Four Architecture Ideas for Gated DeltaNet–Attention Hybrids

Jihoo Lee
Seoul National University

November 2026

Abstract

I propose four architecture ideas for language models that mix Gated DeltaNet layers with softmax attention: a depth mix read only where a block of layers begins; rotary positions whose distance is capped in training and at inference, with the deepest attention layer left without positions; separate depth mixes for what a Gated DeltaNet layer writes and for what it reads; and grid key–value sharing, which at the cache and cost of grouped-query attention builds $r \times c$ distinct keys from $r + c$ cached parts. I tested them, together with two published methods, on models of 60M and 200M parameters with one to three seeds per model. At 60M, over three seeds, the first three ideas and the two methods together lower bits per byte beyond the seed noise and about double multi-key retrieval; the block-entry depth mix carries most of the retrieval gain, and the capped positions largely remove a retrieval failure beyond the training length that, at 16k, I traced in one model to a single rotary frequency. At 200M the composition keeps the lowest bits per byte over two seeds, but retrieval varies so much between seeds that whether the Gated DeltaNet mixes cost retrieval is open. Grid sharing lost no bits per byte against grouped-query attention at the same cache: at 200M it was lower in both seeds, though inside the seed spread, with retrieval within the noise. It redistributes sharing at equal cache and does not replace cache: with a quarter less cache it trailed grouped-query attention, most on multi-key retrieval. The experiments are small, so for each idea I give the reasoning that led to it, what the evidence does and does not show, and the questions still open.

Contents

1 Introduction **2**

2 Setup **4**

3 Block-entry depth mixing **6**

4 Capped RoPE with a NoPE layer **9**

5 Separate write and read mixes for GDN **13**

6 Grid KV sharing **17**

7 Other components (TST, tied embeddings) **21**

8 Composition and scaling **23**

9 Negative results **26**

10 Other small experiments **27**

11 Limitations **28**

12 Background of this work **28**

Acknowledgements **28**

References **28**

A Noise statistics **32**

B Data sources and licences **33**

C Looped models **34**

D Further details **34**

1 Introduction

I set out to build a language model of about 3.8B parameters for agentic and coding work: a hybrid with three Gated DeltaNet (GDN) layers [45] for every softmax-attention layer. Such a model reads long contexts, files, logs and tool output, and must find particular items in them, so I watched two things at once: the loss on held-out text, and retrieval far beyond the training length. Before training it, I tested changes to the architecture on models of 60M and 200M parameters.

In this report I propose four architecture ideas (Figure 1):

1. *Block-entry depth mixing* (Section 3): a learned softmax mix of the embedding and the outputs of earlier blocks, read only where a block of layers begins.
2. *Capped RoPE with a NoPE layer* (Section 4): rotary positions whose relative distance is capped at 2,048 in training and at inference, with the deepest attention layer left without positions.
3. *Separate write and read mixes for GDN* (Section 5): what a GDN layer writes into its state reads one depth mix of earlier blocks, and its query reads another.
4. *Grid KV sharing* (Section 6): the query heads of an attention layer sit on an $r \times c$ grid, and head (i, j) uses the concatenation of row i's and column j's cached parts as its key and value, so $r + c$ parts give $r \times c$ distinct keys.

The first three, together with two published methods I also tested, token-superposition training [32] and tied embeddings [33], make up the architecture I call F (Figure 2). Grid sharing came last; I tested it on F without the GDN mixes.

What I have seen so far is this. At 60M, over three seeds, F has lower bits per byte than the base, beyond the seed noise, and about twice its multi-key retrieval, and block-entry mixing carries most of that retrieval gain (Section 8.1). The cap largely removes a retrieval failure beyond the training length that, at 16k and in the one model I traced it in, came almost entirely from a single rotary frequency. The GDN mixes lowered bits per byte in two-seed comparisons at 60M and at 200M (in the three-seed 60M set, removing them costs bpb within the seed noise); whether they cost retrieval is open, because at 200M the two seeds of F lie further apart on retrieval than F lies from F without the mixes. Grid sharing showed no difference at 60M that the seeds agree on; at

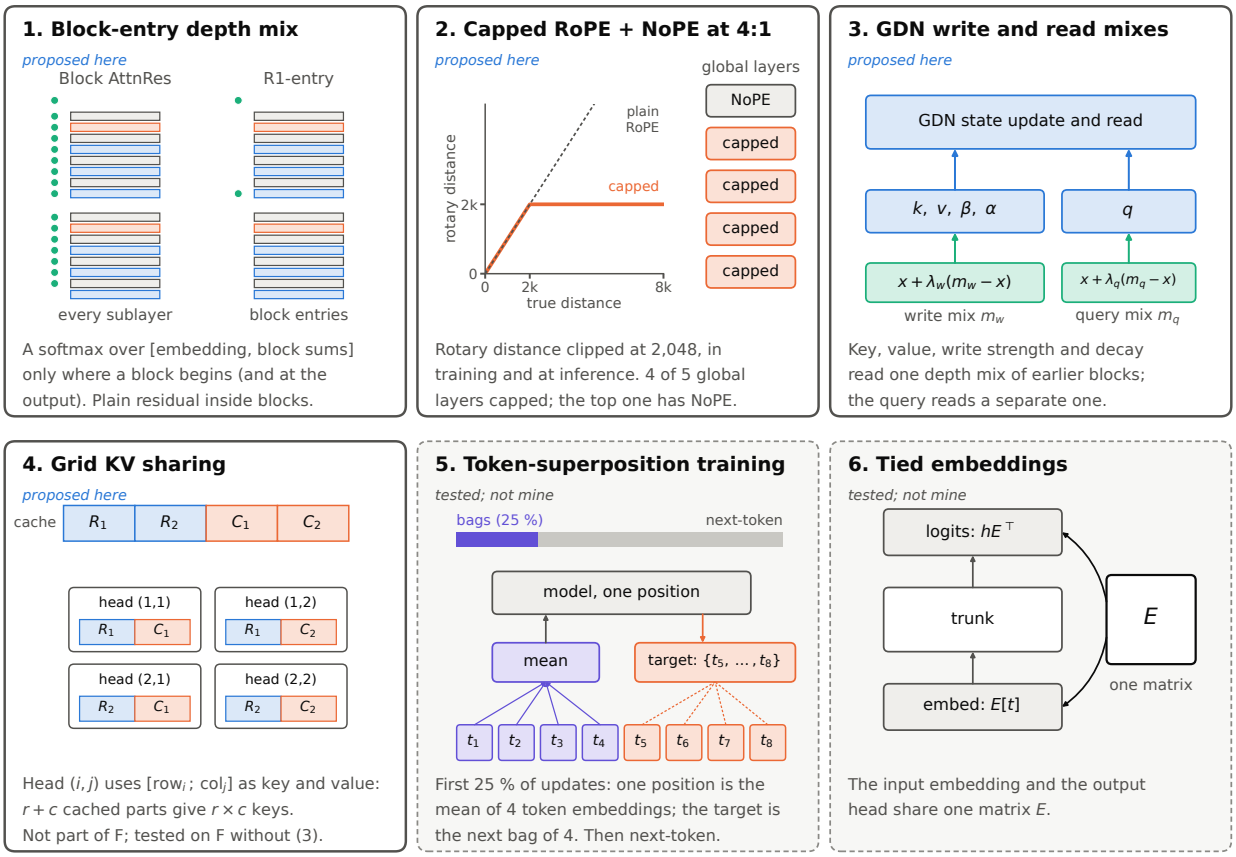


Figure 1: The four ideas proposed here (solid frames) and the two published methods I tested with them (dashed frames). (1) Block-entry depth mixing: a learned softmax over the embedding and earlier blocks' outputs, read only where a block begins and at the output, instead of before every sublayer as in Block Attention Residuals. (2) Capped rotary positions: relative distances beyond 2,048 are treated as 2,048 in training and at inference; the deepest global attention layer has no position code. (3) GDN write and read mixes: the key, value, write strength and decay of each GDN layer after the first read one depth mix of earlier blocks, and the query reads its own. (4) Grid KV sharing: head (i, j) uses row i's and column j's cached parts as its key and value. (5) Token-superposition training: the first quarter of training predicts bags of four tokens from averaged embeddings. (6) Tied embeddings. Pieces 1–3, 5 and 6 make F; piece 4 was tested on F without piece 3.

200M, at the same cost as two KV heads, it lost no bpb and was ahead in both seeds, though inside the seed spread, while its retrieval read lower, within the noise. With a quarter less cache than four KV heads it trailed them, most on multi-key retrieval: the grid redistributes sharing at equal cache and does not replace cache.

Sections 3 to 6 take one idea each, in five parts: the motivation, the design, the results, the open questions, and the related work I found after I had the idea. Section 7 covers the two published methods, Section 8 the composition and its scaling tests, Section 9 the negative results, Section 10 other small experiments, and Section 11 the limitations. Why the report also covers methods that are not mine and lines that went nowhere is told in Section 12.

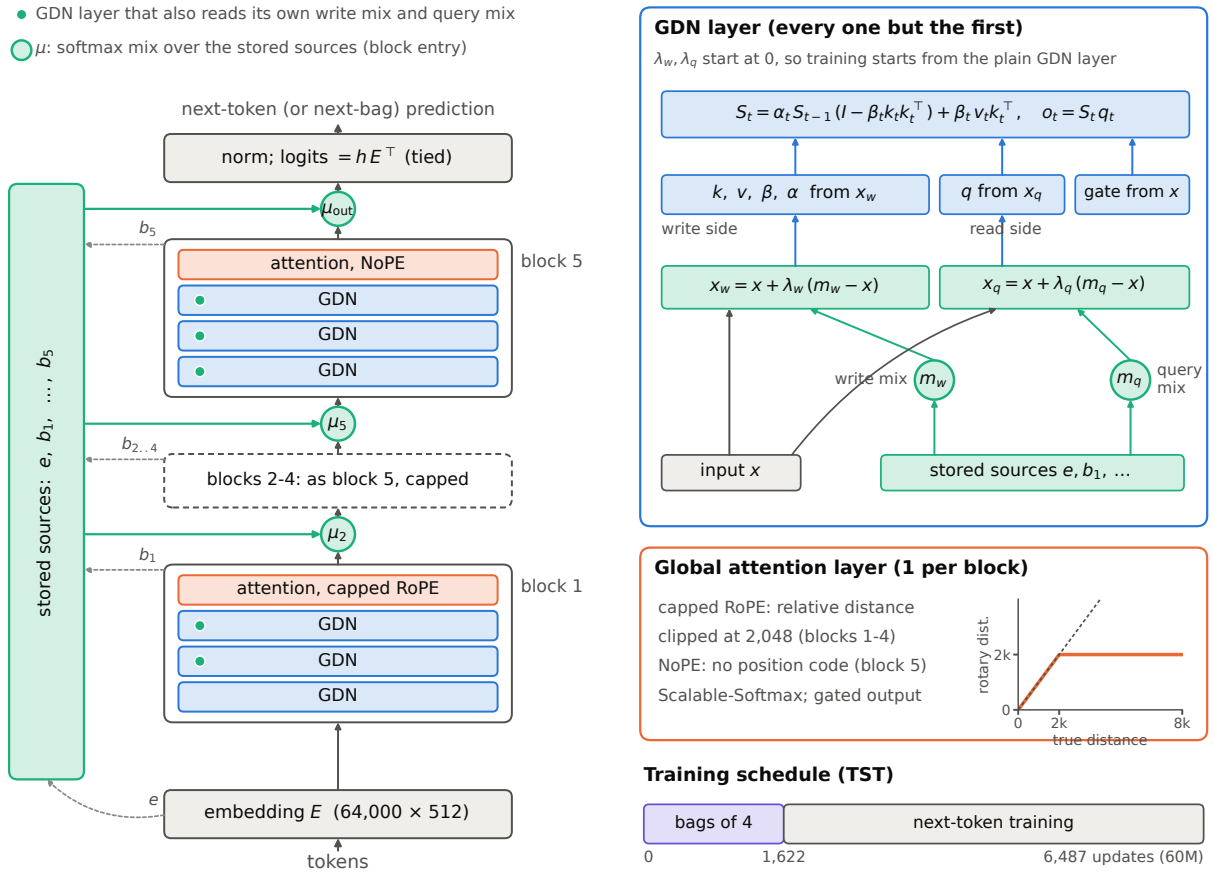


Figure 2: F at 60M. Left: five blocks, each three GDN layers and one global attention layer (feed-forward sublayers not drawn). Block 1 reads the embedding; blocks 2–5 and the output read a softmax mix μ over the stored sources: the embedding e and each finished block's sum of sublayer outputs b_n. Inside a block the residual is the plain sum. Right, top: a GDN layer, whose write inputs and query read separate depth mixes, each blended into the layer input by a scalar that starts at zero (Figure 7 draws it larger). Right, middle: the global attention layers, with the cap on rotary distance and the deepest layer without positions. Right, bottom: the training schedule. At 200M the layout is seven blocks with six capped and one position-free global layer; the 4B draft configuration has ten blocks with eight and two.

2 Setup

The trunk. The base model repeats a block of three GDN layers and one global softmax-attention layer, the 3:1 pattern of recent hybrids [34]. Each layer has a mixer and a SwiGLU feed-forward sublayer [38], with an RMS norm before and after each [17]. The global layers have one key–value head, a sigmoid gate on their output, a norm on queries and keys, and rotary positions on a quarter of each head's dimensions. In the base, the rotary distance is capped at 2,048 in the lower three global layers and the top two have no positions, the setting Section 4 arrives at in its third step. The 60M model has 5 blocks of width 512 (63.7M parameters without the embeddings) and the 200M model 7 blocks of width 768 (208M); the vocabulary is my own byte-level BPE of 64,000 tokens. The 4B draft configuration has 10 blocks of width 2,560; no model of that size has been trained.

Attention scale. Each global layer multiplies its attention logits by $s_h \ln n$, where n is the number of positions the query attends to and s_h is a learned scale for head h (Scalable-Softmax [27]); s_h starts at $1/\ln 8{,}192$, so the factor starts at one at the training length. I added it after a diagnostic on 9 October. Models trained on 8k rows without it lost most single needles beyond 8k, and scoring the same checkpoints, with no training, with their logits multiplied by $\ln n/\ln 8{,}192$ beyond the training length brought most of them back. Scaling only at inference would make inference differ from training, so I built the length-dependent scale into training, where it runs the same way at inference; it cost nothing measurable in bpb or step time. Every model from Section 3 on uses it, the base included, apart from the pure-attention check of Section 7.1 and the plain-softmax comparisons of Section 4; two early screens in Section 10 and the first two comparisons of Appendix C predate it. Apart from those plain-softmax comparisons of Section 4, I did not test whether the results hold with plain softmax.

Training. All runs use Muon [15] on the hidden matrices and AdamW on the rest, a peak learning rate of 0.003, and a schedule that warms up, holds and decays linearly over the last 20% of updates, on rows of 8,192 tokens. A 60M run is 6,487 updates of 24 rows, about 20 token positions per non-embedding parameter; a 200M run is 7,996 updates of 64 rows, in two stages, the second on a smaller set drawn from the first. The learning rate was set at 60M and not retuned at 200M. The training set has 5.15 billion tokens of open web text, PDF text, code, mathematics, science and agent traces (Appendix B), and the model trains on ground-truth tokens only, with no teacher logits.

What I measure. The loss number is bits per byte (bpb), averaged over four held-out sets: web text, code from held-out repositories, the assistant turns of held-out agent traces, and English Wikipedia. For long contexts I use multi-key retrieval: a document hides 4 or 16 key–value pairs and the model must return the value of one key, scored by exact match at 8k, 16k, 32k, 64k and 128k tokens; the mean over these ten cells is *retrieval* in this report, and only the 8k cells are inside the training length. For in-context copying I use the loss on the second copy of a repeated random token sequence. The 200M models are also scored on harder recall sets: the recall tasks of Based [3] (SWDE, FDA and SQuAD completion) and a subset of RULER [11] within the training length, with the needles of three of its six tasks hidden in my own held-out documents (*RULER subset, open haystack*; Appendix B says why). Every benchmark set I score is publicly downloadable; I report scores, never redistribute items, and leave out items that overlap the training data.

Seeds and noise. The final 60M ablation set has three seeds per model on one code snapshot; the other 60M comparisons have two seeds or one, in about equal numbers; at 200M every model has one or two seeds. I call a difference *beyond the seed noise* when it exceeds 2.87 standard errors of the seed-to-seed spread and *within the seed noise* otherwise (Appendix A gives where 2.87 comes from and how often it passes by chance). For bpb the spread is small: at 60M the standard deviation of a model's bpb across seeds is 0.00146. Retrieval is far noisier: its run-to-run standard deviation is 0.076 at 60M and about 1.6 times that at 200M, so with two seeds per model only differences above about 0.2 at 60M, and about 0.35 at 200M, stand out. A difference within the seed noise is not "no effect", one seed is not a spread, and a 60M result is not a 200M result. F's training path was checked for leaks of future tokens (Appendix D.1).

3 Block-entry depth mixing

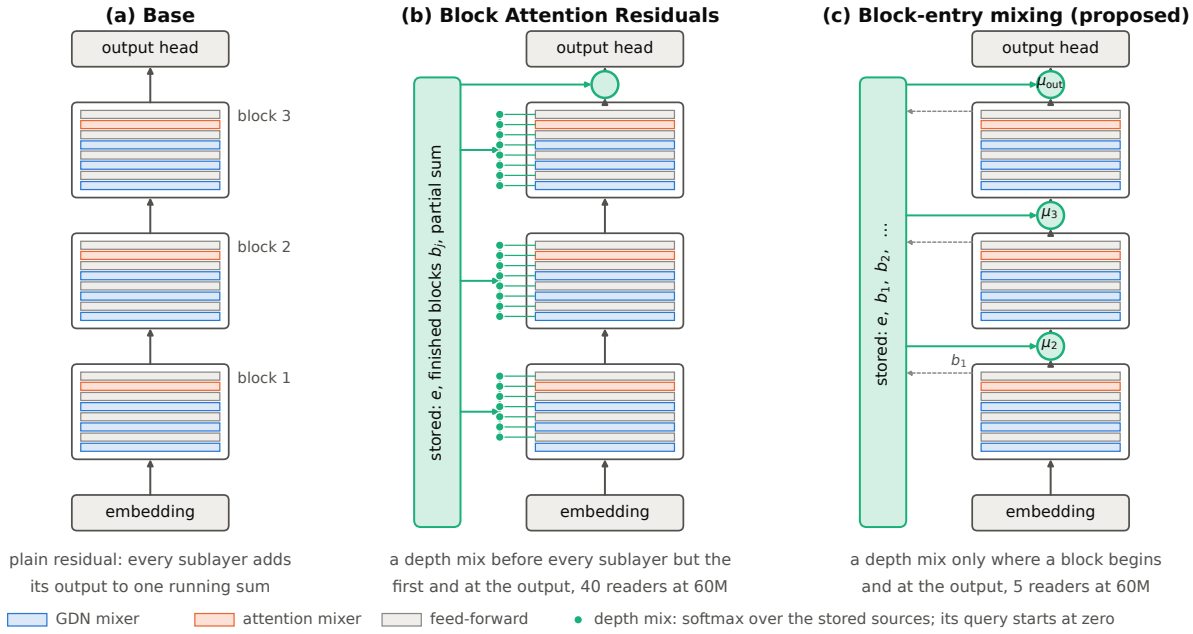


Figure 3: Where each model reads depth, drawn for three blocks of the 3:1 hybrid (the 60M model has five). (a) The base: a plain pre-norm residual stream. (b) Block Attention Residuals: before every sublayer but the network's first, and before the output head, a softmax mix over the embedding e, the sums of the finished blocks b_j and the current block's partial sum. (c) Block-entry mixing: the same kind of mix, read only where a block begins and before the output head; inside a block the residual is the plain sum, and each finished block's sum b_j joins the stored sources.

3.1 Motivation

The starting point is a known property of pre-norm residual streams: every layer adds its output to one running sum with weight one, so the sum grows with depth and each later layer's output is a smaller share of what the next layer reads [18]. My 60M base shows it: on web text, the size of the stream grows from 1.1 at the first layer to 12.1 at the last. If that dilution costs the model something, letting each layer choose how much of the earlier layers it reads should help.

The cheapest test was two learned scalars per layer, $x \leftarrow a\,x + b\,x_0 + f(x)$, with x_0 the embedding, as in the modded-nanogpt speedrun [16]. It lowered bpb beyond the seed noise, and taking the scalars apart showed where the gain sat: re-injecting the embedding alone kept about two thirds of it, while rescaling the stream alone, in one seed, gave a gain within the seed noise (Table 1). I read this as later layers getting the token identity back, a routing effect, more than an undoing of the dilution. A literature check, made when I launched the arms that took the scalars apart, added that static per-layer scalars shrink as models grow, while normalised, convex mixing over depth persists, which pointed to a reader that mixes the embedding and earlier layers with weights that sum to one. Block Attention Residuals do exactly that [18]: layers are grouped into blocks, each finished block's outputs are summed into one source, and before every sublayer a reader takes a softmax over the embedding, the finished blocks and the current block's partial sum, with weights from a learned query that starts at zero (Figure 3b). The paper reports a loss gain that holds from 194M to 528M active parameters on a 3:1 linear-to-full hybrid like mine. With one hybrid period as the block, which gives 40 readers at 60M (I call this R1), it lowered bpb at least as much as the scalars

did and more than doubled 16-key retrieval at 8k, but its readers read 160 stored vectors per token, and on my compiled but unfused code a training step took about a quarter longer than the base's.

That raised the question behind the idea: does the gain need all 40 readers? If the gain is mostly routing, handing each block a fresh mix of the embedding and the earlier blocks, then one read where a block begins should carry most of it. A block of my hybrid ends with an attention layer, so its boundary falls right after the layer that has just looked across the whole context, which seemed a natural place to re-mix. The scalar arm had also cut the stream at exactly those points: its learned scale a fell to about 0.45 after each global layer and rose above one in the next. So I kept only the reader at each block's entry and the one at the output, with the plain residual inside each block: 5 readers instead of 40 (Figure 3c).

3.2 Design

Block $n \geq 2$ starts from

$$h_n^{\text{in}} = \sum_{j=0}^{n-1} \text{softmax}_j(q_n^\top \, \text{RMSNorm}(b_j)) \, b_j, \tag{1}$$

where b_0 is the embedding, b_j for $j \geq 1$ is the sum of block j's sublayer outputs, and q_n is a learned vector that starts at zero, so each block starts by reading the plain mean of its sources. Inside a block the layers add to the stream as usual; block 1 starts from the embedding, and one more reader over $b_0, \ldots, b_N$ feeds the final norm. I expect it to fit any model built from repeated blocks, where in a hybrid the natural block is one period of the layer pattern; I tested it only in this hybrid. Its parameters are one vector and one norm gain per reader, and its memory cost is the stored sources, one vector per block and token. Its cost, three ways. Analytically, a reader scores each of its sources against its query and adds them up with the softmax weights: at 60M the five readers read 20 stored vectors per token, against 160 for R1's forty, which by an exact count is 0.046% of the trunk's forward FLOPs (R1's mixes: 0.35%). On my compiled but unfused code, a training step took about 3% longer than the base's ($\times 1.026$); like any depth mix, its time is set by memory traffic more than by arithmetic, since each reader reads its sources twice. A fused kernel for the reads made the step 0.98 times as long as the unfused path, but it was never timed against the base in the same job.

3.3 Results

At 60M, over two seeds, block-entry mixing kept a little over half of R1's bpb gain, a share that two seeds measure only roughly, and all of its lift in 16-key retrieval at 8k (Table 1). Its weak spot was the single needle at 128k, where it read below the base.

With the reads cheap, I tried to make each one smarter, and neither attempt worked. A reader whose query depends on the token lost on both counts: its bpb was worse than block-entry mixing's and its retrieval lower at every length. Splitting each reader into eight heads with their own softmax over depth, as in Multi-Head Attention Residuals [25], traded one for the other: it gained a little bpb but lost 16-key retrieval at every length. Each was one seed, and the two bpb differences stand out only against the small seed noise I estimated at the time; against the noise of the final ablation set (Appendix A) neither would. My intuition, with no measurement behind it, is that the eight-head reader fits the loss by sharpening the mix toward recent sources, at the expense of the long-range information a lookup needs. The eight-head reader and the GDN write mix of Section 5 showed that a change can buy bpb with retrieval, and from 17 October on I did not accept that trade in any architecture change, because the model is meant for long agentic contexts.

The strongest evidence is the final 60M ablation set, three seeds per model on one code snapshot (Section 8.1). There block-entry mixing beats the base on both bpb and retrieval beyond the seed

Table 1: The path to block-entry mixing at 60M (FINAL). Readers: depth mixes per model. Seeds: the seeds behind the bpb value. bpb: four-set bpb at the last update, the mean over seeds (lower is better). 16-key retrieval at 8k: exact match (the base over five seeds). Needle at 128k: single-needle exact match, one value per seed (the base: seeds 1 and 2). The step time relative to the base, measured on my compiled but unfused code, is in the row label. R1 is Block Attention Residuals with one hybrid period as the block; the last two rows change block-entry mixing's readers. –: not measured.

Model	Readers	Seeds	bpb	16-key, 8k	Needle, 128k
Base	–	3	0.8222	0.160	0.850, 0.770
Residual scalars a, b	–	3	0.8139	–	–
embedding re-injection only (b)	–	2	0.8165	–	–
stream rescaling only (a)	–	1	0.8209	–	–
R1 (step ×1.23)	40	2	0.8119	0.425	0.840, 0.690
Block-entry mixing (step ×1.03)	5	2	0.8163	0.463	0.810, 0.520
query from the token	5	1	0.8214	0.230	0.390
eight heads per reader	5	1	0.8144	0.150	0.820

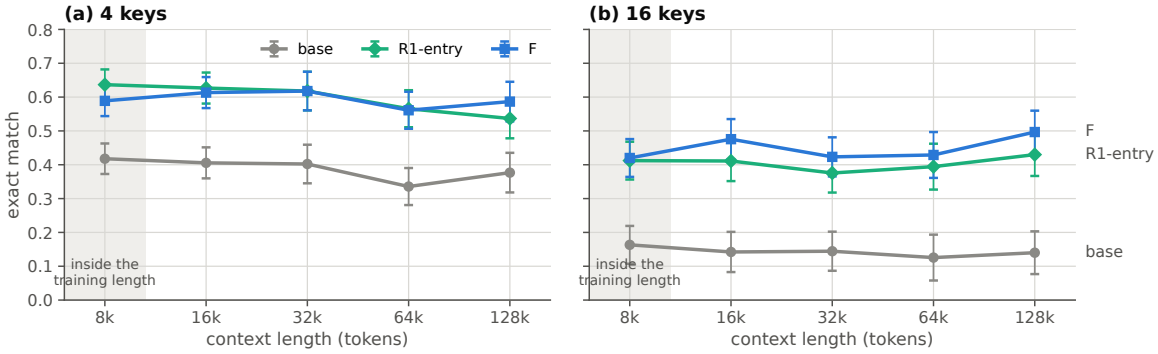


Figure 4: Multi-key retrieval by context length in the final 60M ablation set: the base, the base with block-entry mixing (R1-entry) and F; three seeds each, the mean; the error bars are the cell's seed standard deviation pooled over the ten models of the set, divided by $\sqrt{3}$, the same for the three lines. The shaded cells are inside the 8k training length. Block-entry mixing raises the mean in every cell, beyond the cell's own noise in four of the ten; F adds little on top of it.

noise, and its retrieval is higher at every context length (Figure 4). Inside F, leaving it out costs bpb and retrieval, both beyond the seed noise, the retrieval loss on my standing estimate of the noise though not on the spread of this set alone (the model without it also lacks the GDN mixes, which read its sources); it carries most of F's retrieval gain over the base (Table 12).

At 200M the evidence is thin: one run of block-entry mixing against two runs of the base (Table 3). Its bpb is lower than the base runs', but by less than twice the gap between the two base runs, so it cannot be told from seed noise; its retrieval is higher than both base runs', by an amount within the run-to-run noise for one run against two.

3.4 Open questions

The main question is whether the gain survives scale. For bpb that needs block-entry mixing and the base at 200M and at a larger size with three or more seeds each. For retrieval it needs more seeds still: for a gain the size of the 60M one to clear 2.87 standard errors at the 200M spread, each

model needs five seeds, and for a gain half that size eighteen (Appendix A). Whether block-entry mixing costs field extraction, which one recall set at 200M hints at (Appendix D.4), needs its own test: a field-extraction set scored on several seeds of models with and without block-entry mixing, checking whether the misses are values copied from neighbouring fields. Finally, the cost should be measured with a fused kernel at a larger size: the paper on Block Attention Residuals reports a training overhead under 4% with pipeline parallelism, and negligible without it, and my fused kernel for the block-entry reads ran at 0.98 times the unfused step at 60M, but no single job compares it with the base.

3.5 Related work

I found the designs below after I had the idea. I did not find this exact design in prior work. The closest is the "block" mode of a public re-implementation of Attention Residuals [29], which also reads once at each block boundary; it mixes the stream states left after each block and replaces the stream with the mix, where mine stores each block's raw sum of sublayer outputs and keeps the residual, and it reports no results. DenseFormer's periodic depth-weighted averaging also reads earlier layers only every few layers, with fixed learned scalar weights rather than a softmax over normalised sources [31].

4 Capped RoPE with a NoPE layer

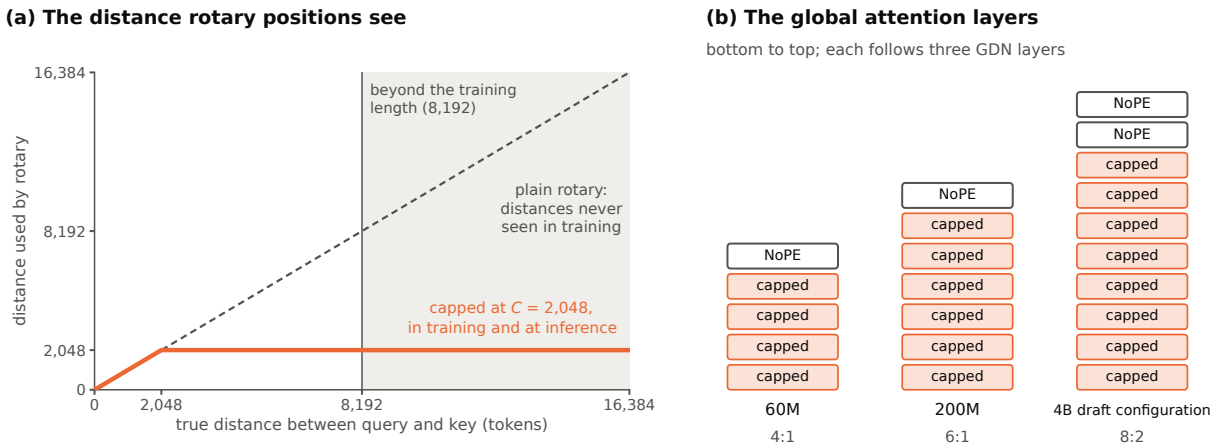


Figure 5: (a) The relative distance a capped rotary layer uses: true distances up to $C = 2{,}048$ as they are, every longer distance as C, in training and at inference. Plain rotary meets distances beyond the 8,192-token training length only at test time. (b) The global attention layers of F at each size, bottom to top: the deepest has no position code (NoPE; in the 4B draft configuration the deepest two) and the others are capped. The base and block-entry mixing use three capped layers and two NoPE layers at 60M. No model of the 4B draft configuration was trained.

4.1 Motivation

This idea started from a measurement in my own runs. At 60M, in a model trained with Scalable-Softmax, single-needle retrieval beyond the 8k training length was not monotone in length: it was less bad at 32k than at 16k and 64k, while the same recipe with plain softmax fell steadily with length. Placing needles at fixed distances showed why: retrieval rose and fell with the needle's

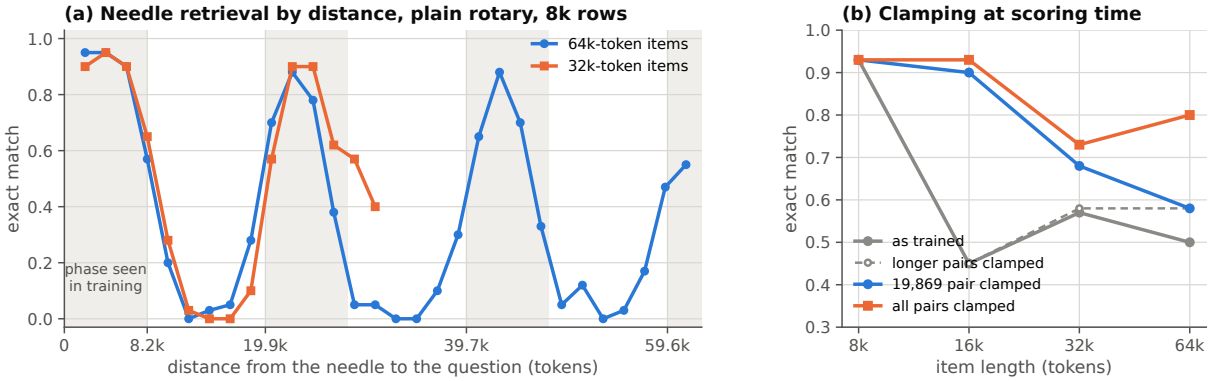


Figure 6: (a) Single-needle exact match against the needle's distance from the question, for a 60M model with plain rotary positions trained on 8k rows; 40 items per point. Shaded: distances at which the rotary pair of wavelength 19,869 tokens has a phase it met in training (distance modulo 19,869 at most 8,191). Retrieval follows that pair's phase, not the length of the document. (b) The same model scored with the distance clamped at 8,191 in chosen rotary pairs, at scoring time only; 60 items per length. Clamping the 19,869 pair alone recovers 16k; at 64k the longer pairs matter too.

distance from the question, with a period of about 20,000 tokens (Figure 6a). The explanation is a known property of rotary positions [43]: dimensions whose wavelength is longer than the training length never complete a turn in training, so at longer distances they produce phases the model has never seen [24]. One rotary pair of my global layers has a wavelength of 19,869 tokens, and 8k rows cover only 41% of its turn. Clamping that pair's distance at scoring time, with no training, doubled single-needle exact match at 16k; clamping the longer pairs alone did nothing at 16k (Figure 6b). So if far distances stopped producing unseen phases, the global layers should keep working beyond the training length. There are two ways to get there: remove the position code (NoPE), or bound the distance it sees. NoPE in all five global layers read needles far better than rotary beyond 8k but cost bpb at 8k, in one seed, and a recent GDN hybrid reports that its NoPE variant ends generations much less reliably after post-training, which bpb cannot show [35]. The other way keeps rotary but treats every relative distance beyond a cap C as C, in training and at inference: local order is kept, and no distance at test time gives a phase training did not see. The intuition is that exact positions matter only nearby. A key 10,000 tokens back and one 10,001 back carry the same meaning for the query; plain rotary tells them apart, while the cap treats all far positions as the same. I started near $C = 2{,}048$, a quarter of the training length. At that cap the far setting gets trained, since 56% of the position pairs in a document that fills an 8k row lie beyond 2,048, while local order is kept over a useful range; only $C = 2{,}048$ was ever run. Each setting then turned out to have something the other lacked, which led to mixing them by layer.

4.2 Design

Rotary positions on a quarter of each head's dimensions, with the relative distance capped at $C = 2{,}048$ in training and at inference, and no position code in the deepest global layer, the deepest two in the 4B draft configuration (Figure 5). At 60M that is four capped layers and one NoPE layer (4:1); the 200M model has six and one, and the 4B draft configuration eight and two. It fits models with a few global attention layers that must work beyond their training length, hybrids especially, since their recurrent layers already see local order. It adds no parameters. I compute the capped

attention as two passes, one over the keys within the cap with their true distances and one over the farther keys at distance C, merged by their log-sum-exp. Each key is still scored once, so the attention arithmetic is that of plain rotary; the cap adds one more rotation of each query and the merge of two partial results for each query and head, a few thousand operations per token and layer against about eight million for the attention itself at the 8k training length. On my compiled but unfused code, with the two passes as two calls of the compiled attention, capped rotary in all five global layers took 0.5% to 1.7% longer per training step than plain rotary, with no extra FLOPs. Moving from 3:2 to 4:1, which turns one NoPE layer into a capped one, measured $\times 1.023$; one changed layer measuring more than five shows that the step-time noise in these measurements is about 2%. No fused measurement of the cap exists.

4.3 Results

The evidence came in four steps (Table 2).

The cap keeps bpb and the needles. Over three seeds at 60M, capped rotary in all five global layers matched plain rotary's bpb within the seed noise and held single needles beyond 8k much better than plain rotary did.

NoPE is stronger far away; the cap is better on bpb. After three short stages on 16k, 32k and 64k rows, capped rotary still had the lower bpb, while NoPE everywhere had slightly better needles at 64k and a much lower loss on a multi-fact task that asks for several facts hidden in one long document, one seed each; neither gap closed.

Mixing by layer. Capped rotary in the lower three global layers and NoPE in the top two (3:2) kept capped rotary's bpb and gave the lowest multi-fact loss of the three settings at every length, in one seed.

One NoPE layer raises retrieval. Later, on the trunk with block-entry mixing and two seeds each, a single NoPE layer (4:1) matched 3:2 on bpb and raised the multi-key retrieval mean by about 0.09; that lead stands out on the seed noise I used then, but not on the larger run-to-run spread I measured later, and of the ten cells only the 4-key cell at 64k rose beyond its own noise. In the final three-seed ablation set, going from 4:1 back to 3:2 inside F gave a lower retrieval mean and the same bpb, both differences within the seed noise.

The cap and the attention scale of Section 2 were settled together. One seed of capped rotary trained with plain softmax shows why: scored as trained, its single needles fell to 0.63, 0.45 and 0.22 at 16k, 32k and 64k, and with the inference-time factor they held at 0.74, 0.72 and 0.77. On this seed the cap holds at long lengths only with a length-dependent scale on the logits, and trained Scalable-Softmax builds that scale in. On plain rotary, Scalable-Softmax trained in had not beaten the inference-time factor beyond the training length; on capped rotary it did, at every trained length, and I fixed it on 12 October.

So I read the evidence for the cap as strong at 60M: three seeds, a mechanism traced in one model, and no bpb cost beyond the seed noise. The evidence for one NoPE layer rather than two is thin: a retrieval lead in two seeds and a lean the same way in three. I read the design this way, and the reading has three parts with different grounds. That the recurrent layers of a hybrid already supply local order is argued in the literature, which drops positions from the attention layers of such hybrids [19, 23]; I have no measurement of it. That the global layers then need positions mainly to tell near from far, which the cap gives them without phases they never trained on, rests on the clamp measurement above. That a NoPE layer helps lookups because it compares content at any distance rests on 4:1 beating 3:2 on retrieval in two seeds; whether that gain comes from distances within the cap or beyond it is undecided (below).

Table 2: The position settings at 60M, in the order of the four steps (FINAL). Four-set bpb (lower is better); single-needle exact match at 64k; the multi-fact loss at 64k (nats per fact token, the mean of six task cells; lower is better); the ten-cell multi-key retrieval mean. Step 2 gives bpb as the difference from plain rotary on the assistant and code sets. –: not measured or not comparable.

Setting	Seeds	bpb	Needle	Multi-fact	Retrieval
1. Base trunk, trained on 8k rows					
Plain rotary	3	0.8217	0.531	1.706	–
Capped, all five layers	3	0.8218	0.768	0.781	–
2. After three stages on 16k, 32k and 64k rows					
Plain rotary	1	0, 0	0.833	0.728	–
Capped, all five layers	1	−0.0014, −0.0023	0.867	0.675	–
NoPE, all five layers	1	+0.0012, +0.0048	0.897	0.496	–
3. Base trunk, trained on 8k rows, seed 1					
Capped, all five layers	1	0.8224	0.793	0.760	–
NoPE, all five layers	1	0.8241	0.740	0.932	–
3:2, capped below and NoPE above	1	0.8224	0.880	0.656	–
4. Trunk with block-entry mixing					
3:2	2	0.81634	–	–	0.517
4:1	2	0.81649	–	–	0.607
F (4:1), ablation set	3	0.80401	–	–	0.5212
F with 3:2, ablation set	3	0.80381	–	–	0.4580

4.4 Open questions

Three things are open. Whether the NoPE layer's gain comes from distances inside the cap or beyond it is not known: in my retrieval sets the near needles come mostly from short documents and the far ones from long documents, so distance and length move together. Needles placed at fixed distances inside long documents of one length would separate them, scored on 4:1, 3:2 and capped rotary alone. Whether one NoPE layer beats two needs more seeds: the lead in two seeds is under half the difference a two-seed comparison can resolve. And whether the design holds where it matters needs a larger model trained on longer rows and post-trained, with the cap swept around 2,048 (no other value was tried) and a test of how reliably generations end. A 60M check of that last point, with prompts from held-out agent traces, found no setting that failed to stop much more often than plain rotary, but at that size and without post-training it clears nothing, and no such test was run on the 4:1 layout or at 200M.

4.5 Related work

I found the designs below after I had the idea. I did not find this design, a cap on the rotary distance applied in training with a NoPE layer on top, in the global layers of a recurrent hybrid, in prior work. ReRoPE uses the same capped position function, but as a patch at inference for models trained with plain rotary [41]; I train with it from the start. Su's hybrid window–full attention puts one NoPE layer on top of position-coded layers [42]; its lower layers attend within a window, where mine attend everywhere with a capped distance. Kimi Linear has no positions in any of its attention layers [19], and Mistral Large 4, a pure attention model, leaves every fourth layer without positions and learns multipliers on its rotary frequencies [13]; neither caps the rotary distance.

5 Separate write and read mixes for GDN

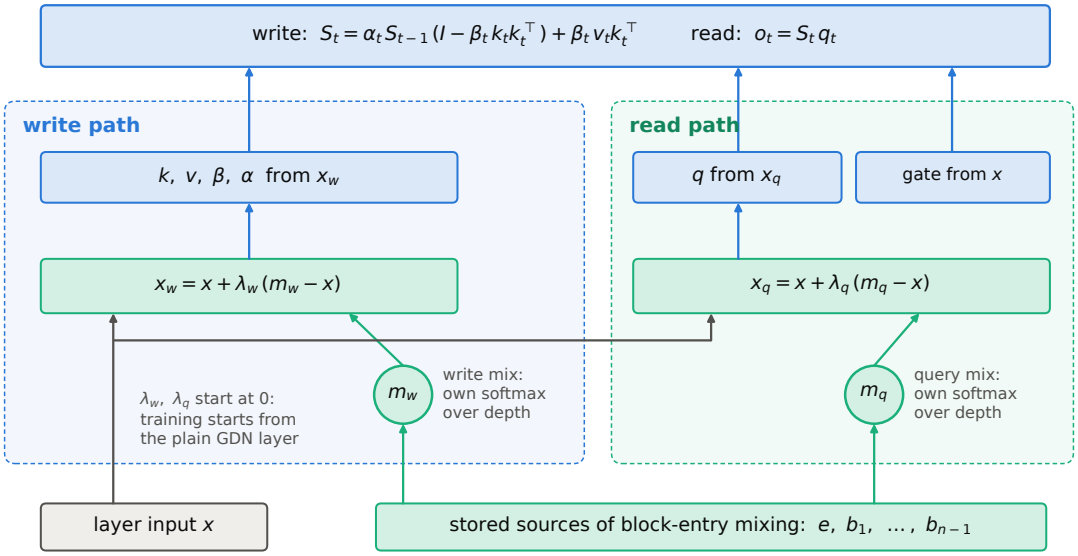


Figure 7: One GDN layer with separate depth mixes for its write path and its read path. The key k, value v, write strength β and decay α are projected from x_w, the layer input blended with the write mix m_w; the query is projected from x_q, the input blended with the query mix m_q. Each mix is a softmax over the sources block-entry mixing stores, with its own learned weights, and each blend starts at zero. The output gate reads the plain input. Every GDN layer but the first has both mixes.

5.1 Motivation

A Gated DeltaNet layer [45] keeps a matrix state S per head and uses its input in two roles. It writes with the key k, value v, write strength β and decay α,

$$S_t = \alpha_t\, S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t\, v_t k_t^\top, \tag{2}$$

and it reads with the query, $o_t = S_t q_t$. The state is the layer's equivalent of an attention layer's key–value cache, and it is small: its size is fixed, however long the context. So my first thought was to let each GDN layer read the states that the other GDN layers hold. On 16 October I wrote down two candidates together. One read the state of the GDN layer below directly, with the reading layer's own query. The other moved the information from other layers to the input side, the design of this section, and had a motivation of its own: keys written from shallower inputs stay close to token identity. The direct read needed a new kernel path and would have roughly doubled the cost of the GDN core, so the input-side form went first, and later that morning I replaced the direct read with a cheap form that reads the state of the layer below only at the chunk boundaries where the training kernel already keeps it. That form gained nothing at 60M, and neither did a second version that read the lowest GDN layer through a learned translator for the query, so I closed it (Table 9). Its diagnosis was that the reading layer's query searched keys written by another layer's key projection, a key space it was never trained on, and a check of the trained model confirmed the mismatch. Mixing on the input side does not have that problem, since each layer's own projections build its keys and queries from the mixed inputs; I read this as consistent with the diagnosis, as an interpretation only. Block-entry mixing (Section 3) already stores the embedding and every finished

block's output, which hold what the earlier layers computed. If the write path of each GDN layer, the four quantities k, v, β and α, drew on those stored sources while the query stayed on the current layer, the layer could store associations built from earlier and shallower information, at the cost of one more mix. I called it *write-shallow, read-deep* (wsl). wsl came first; giving the query a depth mix of its own (qmix) came later, when I tried mixing the query as well (below).

5.2 Design

In every GDN layer but the first, the write quantities read $x_w = x + \lambda_w(m_w - x)$ and the query reads $x_q = x + \lambda_q(m_q - x)$, where x is the layer's normal input, m_w and m_q are two softmax mixes over the stored sources with the layer's own learned queries, and λ_w, λ_q are per-layer scalars that start at zero, so training starts from the plain model (Figure 7).[1] The design fits any recurrent or linear-attention layer with separate write and read inputs, in a model that already stores depth sources; I have tested it only in Gated DeltaNet. For each mix it adds, per layer, a query vector, a norm gain and the scalar. Its cost, three ways. Analytically, each mix scores the stored sources of its layer against its query and sums them, as a block-entry reader does: at 60M each of the two mixes reads 44 stored vectors per token over the fourteen layers, which by an exact count is 0.13% of the trunk's forward FLOPs each and 0.26% together, with the norm and blend each mix adds, or 0.09% each counting only the mixes, as for block-entry mixing. On my compiled but unfused code, a training step with the write mix took about 10% longer than without it ($\times$1.098), and with both mixes 13% to 18% longer. With a fused kernel, the write mix's step came to about 3% over block-entry mixing's fused step ($\times$1.03), compared across two jobs; the fused query mix was not timed against block-entry mixing. I read the gap between 0.26% of the FLOPs and 13% to 18% of the step as the cost of running the mixes as separate small kernels: for the write mix, a fused kernel removed most of it ($\times$1.098 unfused, about $\times$1.03 fused, across two jobs).

5.3 Results

At 60M, over two seeds on the trunk with block-entry mixing, wsl lowered bpb beyond the seed noise, but its retrieval fell (Figure 8). That drop is within the noise, but it was the same pattern as the eight-head depth reader of Section 3: lower bpb, weaker multi-key retrieval. Without weight decay on its scalar the picture was the same. If the write mix stores better associations but the query cannot find them, the query may need its own view of depth; I gave it a second mix with its own weights, so that the read side matches the write side. This qmix kept the bpb gain and brought retrieval back level with no mix at all.

Four further arms (korig, qshare and its two splits, all in Figure 8) asked which write quantities carry the gain and where the retrieval loss comes from. The pattern is this: mixing the key and value buys the bpb gain and leans retrieval down (mixing the value, write strength and decay with the key left unmixed does the same), mixing only the write strength and decay buys nothing (in both splits the query reads the same mix), and the query does best with a mix of its own. These are two seeds each, and on the run-to-run spread I measured after these arms (Section 2 and Appendix A) only retrieval differences above about 0.2 stand out at two seeds, so none of these retrieval differences is beyond the noise on its own; it is their consistent direction that I read.

With token-superposition training (Section 7.1) the bpb gains of qmix and TST added; on retrieval the two together read a little below the trunk with neither, within the noise, although each alone had read level or higher, and two 8k cells fell below the trunk's beyond their own noise

[1] The scalars get no weight decay: in an early arm with such gates, weight decay pulled them back to zero by the end of training, so the new path was measured shut.

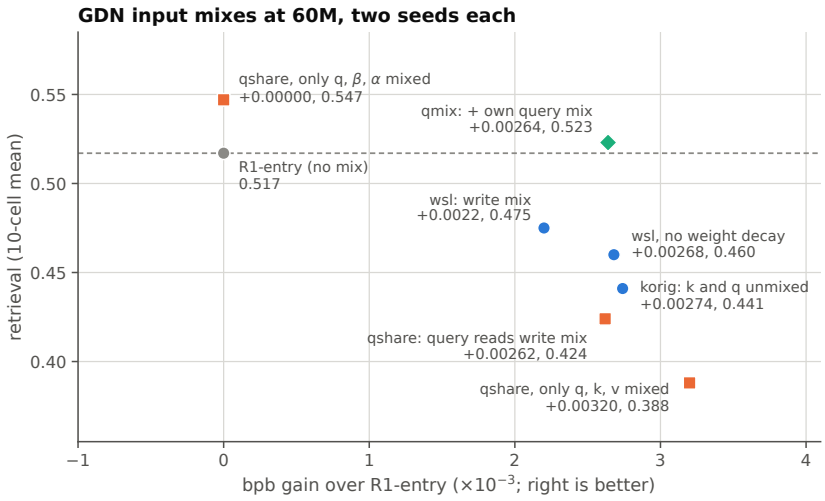


Figure 8: The GDN input-mix arms of this section at 60M on the trunk with block-entry mixing (R1-entry), two seeds each: the bpb gain over that trunk against the ten-cell retrieval mean; each label gives the two values. Blue: the write mix and its variants with the query on the layer's own input; aqua: the query with its own mix (qmix); orange: the query reading the write side's mixed input (qshare) and its two splits, which also mix the query. The dashed line is the trunk without mixes. The arms that mix the key and value gain bpb, and all but qmix lean toward lower retrieval; mixing only the query, β and α gains nothing. At two seeds a retrieval difference must exceed about 0.2 to stand out from run-to-run noise.

(Table 12). In the final 60M ablation set, three seeds, removing both mixes from F costs bpb and raises retrieval, both within the seed noise (Table 12).

At 200M I ran F against block-entry mixing alone, one seed each at first (Table 3 and Figure 12). The first run of F read lower on retrieval than block-entry mixing alone, after the two had been within the seed noise of each other at 60M; with one run each the ten-cell difference was within the run-to-run noise, but two 16-key cells were lower beyond their own noise. The first suspect was TST's data repeat: at 200M the runs with TST read their first-stage data more than once, while the runs without it read only about half of it. The TST-only runs repeat the same way, and none of their 16-key cells fell below block-entry mixing's beyond its own noise, though their mean was lower too; that pointed away from the repeat, but they lack the other pieces and cannot rule it out. F without the two GDN mixes read close to block-entry mixing alone, so the mixes became the main suspect. My hypothesis was that mixing pulls the key toward shallow, token-identity features, so that keys collide in the state, while the value benefits from those features.

Second seeds of both models repeated the bpb gain, outside the spread of the 200M seeds and on every held-out set. Retrieval did not hold still: each model's second seed read far higher than its first, and the model without the mixes was ahead in both seeds, by an amount inside the seed spread.

Three test arms on F, one seed each, tried to separate the key from the value: the key and query unmixed, only the value mixed, and the key and query on a shared mix of their own. All three ended within about 0.001 bpb of F's first seed. Against block-entry mixing alone, the arm with the key and query unmixed lost no retrieval cell beyond its noise, while the value-only arm and the shared key–query arm each lost four or five of the ten cells beyond their noise. If colliding keys were the whole cause, the value-only arm should have kept retrieval; it did not, but these are single runs, the two seeds of F lie further apart than the three arms do, and nothing rests on them. Their

Table 3: The 200M runs: four-set bpb at the last update and the ten-cell retrieval mean, one value per seed. The last three rows are test arms on F. F and TST only read 1.1804 passes of the first-stage data, the base and R1-entry 0.5698. All values shown are FINAL.

Run	bpb	Retrieval
Base	0.717109, 0.716543	0.456, 0.407
R1-entry	0.715834	0.655
TST only	0.7169, 0.716828	0.574, 0.578
F	0.701958, 0.70133[a]	0.500, 0.773
F without the GDN mixes	0.70588[a], 0.70488[a]	0.629, 0.828
F, key and query unmixed	0.70278[a]	0.599
F, only the value mixed	0.70193[a]	0.437
F, key and query on their own mix	0.70297[a]	0.389

[a]The mean of the four set scores, computed from the delivered per-set values.

second seeds, which test whether the value-only result repeats, were running or queued at the time of writing (7 November 2026), with results to follow in a later version.

Two probes of the trained models, the attention the global layers put on the needle and the GDN state's read-out against the value written at the right position, did not track which items a model answered correctly from one seed to the next, so they have not located a cause. At 200M, over two seeds, F keeps its bpb lead over F without the mixes on every held-out set, and its retrieval is within the seed noise of R1-entry's and of F without the mixes. Whether the mixes cost retrieval at this scale is still open.

5.4 Open questions

The direct question is whether the mixes cost retrieval at scale. At the 200M seed spread, the average gap seen so far would need about 29 seeds per model to clear 2.87 standard errors, which is impractical, so the useful tests are different; and since that spread is estimated on only 4 degrees of freedom, the count could be anywhere from about 11 to 235. One is a direct measure of key collisions: for items the model gets wrong, how close the keys written for different needles are in the GDN state, with and without the write mix. Another is the second seeds of the three test arms (Section 5.3), which say whether the value-only result repeats. A third is a larger model, trained on longer rows, where the retrieval cells are further from their floor and ceiling. If the cost is real and sits in the key, mixing only the value and gates while keeping the key on the layer's own input (korig) is the form to test first.

5.5 Related work

I found the designs below after I had the idea, and I did not find this design in prior work. MUDDFormer gives the query, key, value and residual streams of each softmax-attention block their own depth mixes, with weights generated per position [44]; mine groups the key, value, write strength and decay into one write mix and gives the query a second, inside a recurrent delta-rule layer. LIMe lets each attention head draw on the keys and values earlier layers have already computed, through learned routing weights [6]; mine mixes the inputs of a GDN layer's projections rather than earlier layers' keys and values, and splits them into a write side and a read side. DenseRetNet adds earlier layers' keys and values to a later linear-attention layer and leaves its query alone, as wsl does [10]; its layer has a fixed decay and no write strength, and it adds the keys and values of the previous layers, each passed through a small gated projection, rather than a softmax mix of block outputs.

6 Grid KV sharing

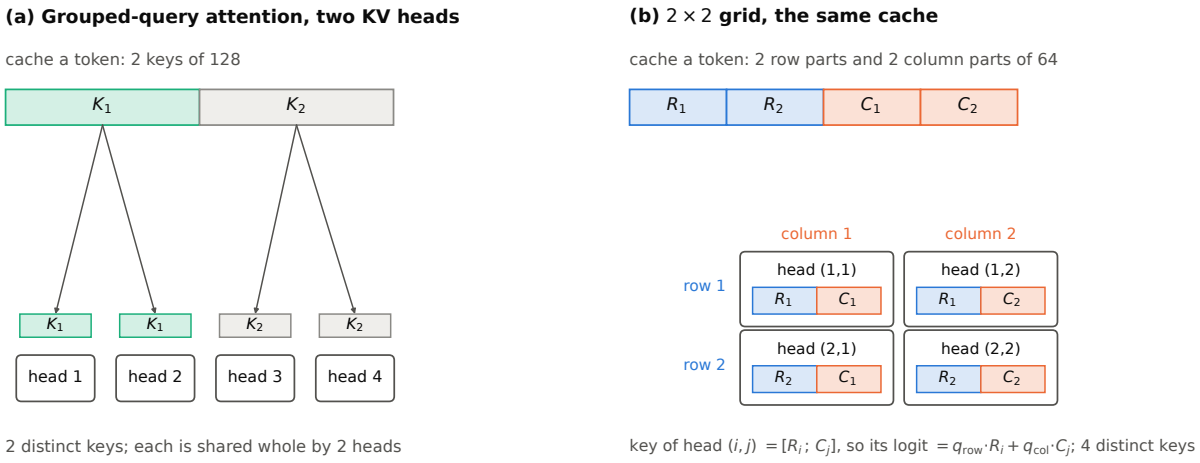


Figure 9: Grid KV sharing against grouped-query attention at the same cache, for four query heads with keys of 128 numbers. (a) Two KV heads: two keys, each shared whole by two heads. (b) A 2×2 grid: two row parts and two column parts of 64; head (i, j) uses $[R_i; C_j]$, so the four heads get four distinct keys, and each head's logit is a row term plus a column term. Values are built the same way.

6.1 Motivation

I started from how grouped-query attention splits the query heads into groups that each get their own key and value [1]. It shares keys too much: every head in a group uses the same key, so the keys lack diversity. The question was how to keep the key–value cache and increase that diversity. Instead of groups, lay the heads on a grid, give each row one matrix and each column one matrix, and let the head at (i, j) use the two joined (Figure 9). For each token the cache then holds one key part for each row and one for each column; the head in cell (i, j) uses the concatenation $[\,\mathrm{row}_i\,;\,\mathrm{col}_j\,]$ as its key, and its value is built the same way. So $r + c$ cached parts give $r \times c$ distinct keys. With parts half a head wide, the cache of n KV heads holds $2n$ parts, enough for an $n \times n$ grid: at the same cache the grid gives n^2 distinct keys where grouped-query attention gives n. If distinct keys are what helps retrieval, the grid should beat grouped-query attention at equal cache. It could also attend differently. Because a grid head's score is the sum of a row part and a column part, after the softmax its attention is the product of two patterns, so a head attends where both its row criterion and its column criterion match; this is my interpretation, and I have not tested it.

6.2 Design

In the runs, heads keep their size of 128 and each part is 64 wide; one key norm acts on the whole 128-wide key, and the 32 rotary dimensions lie in the row part. Against grouped-query attention with the same cache, the grid has the same parameter count and the same starting weights, so the two differ only in how the cache is shared. Neither pattern contains the other. Counted in parts of 64, grouped-query attention pairs the parts in fixed, disjoint pairs, while the grid pairs every row part with every column part; the grid cannot give two groups of heads two unrelated keys, and grouped-query attention cannot give two heads half a key in common. The two meet at one KV head, a 1×1 grid. To compare the patterns I count, for two query heads of a layer, the fraction of their key that is the same cached numbers (1 for one group or cell, $\frac{1}{2}$ for a shared row or column

Table 4: Sharing between the query heads of one layer, for every configuration in this section. Cache is in KV heads of 128 numbers, for the key (the value is the same). A pair of heads shares all, half or none of its key; the average is over all pairs. Arithmetic on the design, not a measurement.

Configuration	Query heads	Cache	Distinct keys	All	Half	None	Average
60M, one KV head	4	1	1	6	0	0	1
60M, two KV heads	4	2	2	2	0	4	1/3
60M, 2×2 grid	4	2	4	0	4	2	1/3
200M, one KV head	8	1	1	28	0	0	1
200M, two KV heads	8	2	2	12	0	16	3/7
200M, 2×2 grid of head pairs	8	2	4	4	16	8	3/7
200M, four KV heads	8	4	4	4	0	24	1/7
200M, 2×4 grid	8	3	8	0	16	12	2/7

part, 0 otherwise) and average it over all pairs (Table 4). At equal cache the 2×2 grids share exactly as much as grouped-query attention, because when every cached number is read by n of the h query heads the average is $(n-1)/(h-1)$ whatever the pattern; the grid shares in halves among more pairs, and no two heads in different cells share a whole key.

The grid generalizes to k axes: axis a holds n_a cached parts, a head picks one part on each axis and joins them, and $\sum_a n_a$ parts give $\prod_a n_a$ distinct keys. When every axis has the same number of parts n, every cached number is read by H/n of the H query heads, so at the cache of n KV heads the heads per part and the average sharing are those of grouped-query attention with n KV heads, for any k; this is the equality of Table 4, and it needs that condition, which the 2×4 grid does not meet. What changes with k is the structure of the sharing, since at equal axis widths two heads that agree on j of the k axes share j/k of their key, and the number of distinct keys the same cache gives: n^k at the cache of n KV heads, so at the cache of two KV heads a $2 \times 2 \times 2$ grid gives each of eight heads its own key, where a 2×2 grid gives four. My intuition, with nothing tested behind it, is that two axes may match the two kinds of information a key carries here, a part mixed with position, which holds the rotary dimensions, and a content part; a third axis has no clear counterpart, so splitting the content further may only add constraints (a $2 \times 2 \times 2$ run tests this; Section 6.3).

The same construction could build the keys of GDN layers, where pairs of value heads now share one key, or the latents of grouped latent attention [46]; both are expectations I have not tested.

6.3 Results

Grid KV came to me after F's composition was fixed; folding it in would have meant rerunning the ablation set, which I had neither the time nor the compute for, so I tested it on its own. My small models all use one KV head, and earlier tests of more KV heads either changed other things at the same time or disagreed between their seeds (Appendix D.2); the grid's 200M runs include two KV heads against one (below). Both the grid and its controls are built on F without the GDN mixes, which has one KV head, and both are also compared with it; that comparison mixes the cache size, the extra key–value weights and the sharing. At 60M the four query heads form a 2×2 grid, against two KV heads, three seeds each (Table 5, FINAL). The grid's bpb was below grouped-query attention's on two of the three data orders and above it on the other; because the three differences do not share a sign, my reading rule calls the comparison SPLIT, with a small mean difference in the grid's favour. Two KV heads against one was SPLIT as well, with a mean difference near zero. On retrieval the three models are close together: the grid passes my rule against two KV heads, and both pass against one KV head. So the prediction was not borne out at this size: four distinct

Table 5: Grid KV sharing at 60M (FINAL): four-set bpb at the last update by seed, and the ten-cell retrieval mean over the three seeds. The grid and its control have the same cache; one KV head has half of it.

Model	Seed 1	Seed 2	Seed 3	Retrieval
One KV head	0.807126	0.806128	0.805829	0.590
Two KV heads	0.80536	0.80498	0.80893	0.601
2×2 grid	0.80355	0.80790	0.80225	0.608

keys gained almost nothing on retrieval over two, and two almost nothing over one; with three seeds and four query heads this cannot show that distinct keys do not help.

The seeds of both new models spread wider than those of one KV head: their final bpb ranges over 0.0056 for the grid and 0.0039 for two KV heads, against 0.0013. So I looked for instability in the training logs. There was none: no restart, no non-finite value, and no loss spike that grouped-query attention lacked. The configurations are the same apart from the seeds and the key–value sharing, and within a data order the three models read the same batches. Against its two seed-mates, the grid's second seed moved off at the switch out of TST's token bags, and the third seed of two KV heads during the bag phase; after that neither offset grew. Some one-KV-head arms of earlier 60M sets spread as widely, so I read this as the ordinary variation of a run that settles early into an offset, not as a fault of the grid. The one-KV-head runs trained on an older code snapshot than the grid runs, and the grid's seed variance is 4.14 times that of the ablation set on that older snapshot (one-sided $p = 0.031$). At 60M there are only four query heads; the 200M runs repeat the comparison with eight.

At 200M the eight query heads form a 2×2 grid of head pairs, against two KV heads, two seeds each (Table 6). The grid's bpb was lower than that of two KV heads in both seeds, by an amount inside the spread of the 200M seeds ($t = -1.47$). Its retrieval was lower in all ten cells, each within its own noise, and the ten-cell mean was lower by an amount within the run-to-run noise ($t = -1.00$): the retrieval check passes, but leans toward two KV heads, and most of the gap is the grid's first seed, whose ten-cell mean sits far below its second. Two KV heads against one split between the seeds on bpb, and the two read the same on retrieval. So at the same cost the grid lost no bpb and was ahead in both seeds, while the retrieval prediction was not borne out; with two seeds neither difference can be told from seed noise. A second pair is a negative result. It put the eight single heads on a 2×4 grid, with the cache of three KV heads and eight distinct keys, against four KV heads, with a third more cache and four distinct keys, two seeds each. The grid's bpb was higher than that of four KV heads in both seeds, by an amount inside the spread ($t = +1.94$). Its retrieval failed my check through the 16-key cells at 8k, 16k and 32k, each below its own bar, although its ten-cell mean, 0.652 against 0.789, passed the main test ($t = -1.81$); and on FDA, the field-extraction set of Appendix D.4, four KV heads scored higher in both seeds. Four KV heads also had lower bpb than two outside the spread, in both seeds, so at this size more cache helps. The conclusion is plain: the grid redistributes sharing at equal cache and does not replace cache; with a quarter less cache it trails four KV heads, most on multi-key retrieval. Two further runs were going at the time of writing (7 November 2026), with results to follow in a later version. A third seed of the grid and of two KV heads tests whether the grid's lead in bpb holds over three seeds. A $2 \times 2 \times 2$ grid of single heads, with axis widths 64, 32 and 32 and the same cache as two KV heads, gives each of the eight heads its own key; it tests whether a third axis, which splits the content part in two, helps or only adds constraints.

Table 6: Grid KV sharing at 200M: four-set bpb at the last update and the ten-cell retrieval mean, by seed. Cache is in KV heads of 128 numbers. One KV head is F without the GDN mixes from Table 3.

			bpb		Retrieval	
Model	Cache	Distinct keys	Seed 1	Seed 2	Seed 1	Seed 2
One KV head	1	1	0.705881	0.704882	0.629	0.828
Two KV heads	2	2	0.705193	0.706748	0.728	0.727
2×2 grid of head pairs	2	4	0.704496	0.705512	0.532	0.770
Four KV heads	4	4	0.704149	0.703743	0.856	0.723
2×4 grid	3	8	0.706741	0.704363	0.671	0.633

6.4 Open questions

What the grid runs do and do not test. Three points bear on the grid beyond what my runs settle, and each is of a different kind. *A property of the design*, worked out but not measured: at equal cache the grid has the parameters, the cache and the attention arithmetic of grouped-query attention, since each head forms one dot product of length 128 per cached token, as the sum of a dot product with its row part and one with its column part, and a decoding kernel that loads each cached part once would move exactly the cache; only the addressing changes, plus one sum of squares per part and token for the key norm. So the grid can replace grouped-query attention in any layer with at least two KV heads' worth of cache, and under tensor parallelism, where each device holds some heads and the key–value parts they read so that attention needs no communication [39], it keeps that property when each device holds whole grids: with m KV heads' worth of cache per device, m^2 distinct keys where grouped-query attention has m, and with one KV head's worth nothing changes. I have not written that kernel: the runs assembled each head's key and value for the ordinary attention kernel, so their step times do not test the cost; what the first 200M pair tests is the bpb, and there the switch cost nothing, the grid being lower in both seeds, inside the spread. *An arithmetic fact*: my runs sit where the gap in distinct keys is smallest. At the cache of n KV heads with H query heads, the grid gives n^2 distinct keys, or one per head once n^2 reaches H, where grouped-query attention gives n, so it has $\min(n, H/n)$ times as many; every equal-cache comparison here has $n = 2$ and the ratio 2, the smallest above 1: four keys against two at 200M. In a common large-model setup, 32 to 64 query heads on the cache of eight KV heads, an 8×8 grid gives every head its own key, 32 to 64 against eight, with the same average sharing as grouped-query attention. That regime is untested, and at $n = 2$, with two seeds that catch only large retrieval differences and grid seeds that lie far apart, whether distinct keys help at equal cache is open. *An untested prediction*: a grid head's score is a row term plus a column term, so its attention is the product of a row pattern and a column pattern (Section 6.1); in the layers with rotary positions all 32 rotary dimensions lie in the row part and none in the column part, so a head should attend where a criterion that depends on position and one that depends only on content both match. It can be checked on the trained 200M grids without new training, by comparing each head's attention with the softmax of its row term alone and of its column term alone.

6.5 Related work

I found the designs below after I had the idea, and I did not find this design in prior work: no design I found builds each head's key and value as the concatenation of a part shared along its row and a part shared along its column. The closest is Kronecker product attention, which also indexes heads by a pair (i_1, i_2) and caches one factor per grid row and one per grid column [28]; it multiplies the

row and column factors, so for each of its rank terms a head's score is the product of a row term and a column term, while the grid concatenates them, so the score is their sum. In grouped-tied attention a head's key is its group's half followed by one rotary half shared by all heads [46], which is the grid with a single column for the key; its value is the group's whole tied state. Product key memory counts the same way: it builds $|C| \times |C'|$ keys from $|C| + |C'|$ sub-keys by concatenation, so a key's score is the sum of two half scores [20]; but its keys index a learned table of values chosen by top-k, while the grid's parts are cached for each token and each head attends over all tokens with the key of its own cell. Three other schemes share keys across heads in other ways: tensor product attention writes each token's keys as sums of outer products of a head factor and a feature factor and caches the factors [47]; low-rank key–value attention gives every head one shared key plus a low-rank key of its own, added rather than joined [30]; and multi-matrix factorization attention caches a single key and value for all heads and puts the differences between heads on the query side [12]. None of them has parts shared along crossing rows and columns of heads.

7 Other components (TST, tied embeddings)

The two methods in this section are published; I tested them in my hybrid and kept both in F.

7.1 Token-superposition training

Token-superposition training (TST) [32], published earlier as patch-level training [37], changes only the start of pretraining: for the first fraction of the updates, each input position is the mean of the embeddings of s consecutive tokens, and the model predicts the next bag of s tokens; then training continues as plain next-token prediction. In the paper's Table 1, its untied dense transformers from 270M to 3B parameters, trained for 20,000 steps, end with equal-compute losses 2.18% to 4.70% lower than plain training (my arithmetic on its values), and a bag position costs what a token position costs, so with bags of 4 for the first quarter the run reads 1.75 times the raw text for the same compute. So when the data available exceeds the compute, TST should clearly help. Both papers test softmax-attention models only, so I tested it in my hybrid.

What I found. At 60M, bags of 4 for the first 25% of updates (s4r25) were the best of my first three settings and lowered bpb beyond the seed noise over two seeds, but only by about a quarter of a percent, far below the 2.18% of the paper's 270M model (Table 7); on the trunk with block-entry mixing the same setting read 0.78%, against controls trained on an earlier code snapshot. Most of the gain lands on rare tokens: the token types outside the 10,000 most frequent, under a quarter of the bytes, carry about two thirds of it, and the rarest of them, seen fewer than 10,000 times in training, carry a quarter of it on 6% of the bytes. A form that keeps token inputs and trains only on bag targets lost the whole gain, so the gain needs the bagged inputs: it comes from reading more text through them. Inside F, removing TST costs bpb beyond the seed noise and makes in-context copying worse: the loss on the second copy of a repeated random sequence is higher at all four lengths I score (Tables 12 and 7). The price is a jump in the loss when the bags end: inside F, the training loss first falls below that of F without TST about 600 updates after the switch and stays close to it after that, in 50-update windows of which the first after the switch holds only 28 updates (Figure 10). At 200M, two seeds each, TST's bpb was level with the base's, inside the base runs' own gap, while its retrieval was higher by an amount within the 200M run-to-run noise (Table 3); but the TST runs also repeated part of their first-stage data where the base did not, a confound an analysis put anywhere from nothing to a few thousandths of a bpb. An audit of my code against

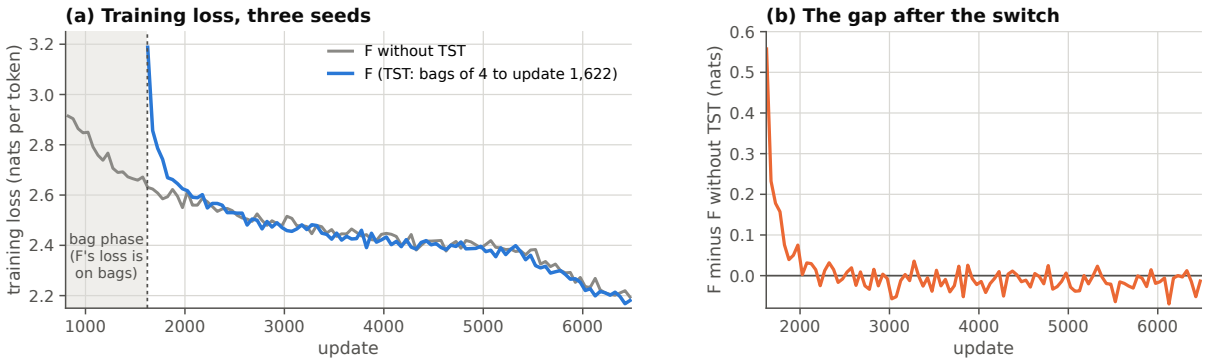


Figure 10: TST's switch inside F, 60M ablation set, three seeds. (a) Training loss of F and of F without TST, in 50-update windows (F's first window after the switch holds 28 updates); F's loss during the bag phase is a loss on bags and is not drawn. (b) The difference after the switch: F's loss starts about half a nat higher and first falls below the other model's about 600 updates after the switch, after which the two stay close; F's held-out bpb lead over it builds up from about update 3,000 and is 0.00463 bpb at the end.

the paper found no implementation bug. On a small pure-attention model with untied embeddings and AdamW, closer to the paper's models but not of their shape (18 layers, not 30, with norms after the sublayers as well), the same code lowered four-set bpb by 1.16% over two seeds with the paper's own setting, between the 1.0% and 1.4% I had fixed beforehand as a hybrid-sized and a paper-sized gain, and by 1.52% with s4r25 in one seed.

Strengths and weaknesses. In my hybrid TST is a real but small gain at 60M that lands on rare tokens, and with a bag size of one the code reproduces plain training bit for bit, so it adds nothing to the recipe beyond the first quarter. Its weaknesses are that the gain in my hybrid is several times smaller than on the paper's models and was not visible in bpb at 200M, and that it reads more raw text, which a data budget must supply or repeat. I cannot yet say why my models gain less: no run separates the optimizer (Muon on my hidden matrices, AdamW in the paper), tied embeddings and the hybrid itself. My intuition, with no measurement behind it, is that the GDN layers already carry a running summary of the context, which makes the bags less useful.

7.2 Tied embeddings

Tying the input embedding and the output head to one matrix is an old idea [14, 33]. With my 64,000-token vocabulary the matrix is about a third of a tied 60M model and 4.4% of the 4B total, so a fair test at 60M has to grow the trunk only by the fraction tying frees at the 4B.

What I found. Tying alone on the base, which removes a quarter of the parameters, was not clearly different from the base's bpb in one seed. Tying with the feed-forward widened from 1,408 to 1,512, which grows the 60M trunk by 5%, about the share of the trunk tying frees at the 4B, lowered bpb beyond the seed noise over two seeds on the trunk with block-entry mixing and kept the two-seed retrieval mean level, though its two seeds differ widely; this is the form I kept, for faster convergence in bpb. The evaluations beyond bpb found an effect I had not looked for: without tying, F copies a repeated random sequence much worse, the largest change of any piece removed, and the extra loss sits mostly on rare tokens. On a fresh random sequence, which cannot be copied,

Table 7: The measurements behind Section 7 (FINAL). bpb is the four-set mean at 60M (lower is better); the copy loss is in nats per token on the second copy of a repeated random sequence.

Measurement	Seeds	Value
Base: bpb	3	0.8222
TST s4r25, s6r25, s6r40 on the base: bpb, seed 1	1	0.82002, 0.82180, 0.82377
TST s4r25 on the base: bpb	2	0.8205
Bag targets with token inputs: bpb against the base	1	+0.0021
Share of the s4r25 gain: types outside the top 10,000; of them, those seen under 10,000 times	2	64% (23.1% of bytes); 25% (6.1%)
s4r25 and the paper's s6r30 on a pure-attention model	1, 2	1.52%, 1.16% lower four-set bpb
At 200M, the data repeat's possible effect on TST's bpb	–	0 to 0.0036
Copy loss at length 256, F and F without TST	3	0.14313, 0.22474
Tied, same trunk, on the base: bpb	1	0.8212
Tied, trunk 5.0121% larger (wider feed-forward), on R1-entry: bpb gain; retrieval	2	0.00553; 0.511 (0.517 untied)
Copy loss at length 256, F and F untied	3	0.14313, 0.36206
Extra copy loss untied, rare and common tokens (four lengths pooled)	3	0.3563, 0.0802

the extra loss shows no such concentration. Removing tying from F costs bpb within the seed noise (Table 12).

Strengths and weaknesses. Tying frees parameters for the trunk and, in my runs, helps the model reproduce rare tokens it has just read, which an agent does constantly with identifiers and file paths; my probes use random token sequences, so that last step is an expectation. Press and Wolf's account, that tied rows of rare words are updated at every step while untied input rows are updated rarely [33], is consistent with the frequency split but not tested by it. The weakness of my evidence is that the tied form I kept also has a 5% larger trunk, and no run separates tying from that width; the only arm tied without widening is a single seed on the base trunk.

8 Composition and scaling

8.1 The 60M ablation set

Each piece was first tested alone, against its own trunk, and small gains measured that way do not have to add up; so the question here is what each piece contributes inside F. To answer it with three seeds per model on one code snapshot, I ran a final set of 60M models: F, the base, the base with block-entry mixing, and F with each piece left out. F's bpb is below the base's and below block-entry mixing alone, both beyond the seed noise, and its retrieval is about twice the base's and within the seed noise of block-entry mixing alone. Two pieces cost bpb beyond the seed noise when left out, block-entry mixing (which takes the GDN mixes with it) and TST, and block-entry mixing also carries the retrieval gain; leaving out 4:1, the GDN mixes or tying stays within the seed noise on both measures, which with three seeds and retrieval's noise is not the same as no effect (Figure 11; the values are in Table 12). The set measures the pieces after I had settled them, so it checks the composition rather than choosing it. Before it, I had stacked the pieces on block-entry mixing two seeds at a time: together they gained a little less than the sum of their gains alone, and the copy-pointer head of Section 9, by far the largest bpb gain inside the stack and the largest

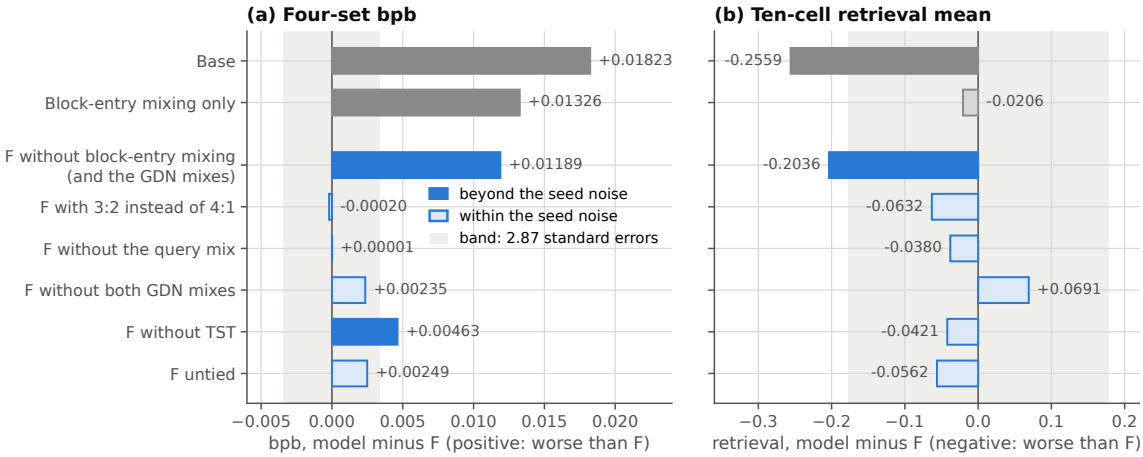


Figure 11: The 60M ablation set, three seeds per model: each model's difference from F in (a) four-set bpb and (b) the ten-cell retrieval mean. Grey: the two reference models; blue: F with one piece left out. Filled bars are beyond the seed noise; the shaded band is 2.87 standard errors of a difference of two three-seed means. Leaving out block-entry mixing also removes the GDN mixes, which read its sources. The values are in Table 12.

retrieval loss, left it. I settled F as the architecture at that point, before its 200M reading; Section 5 tells how the 200M runs reopened the GDN mixes.

8.2 F at 200M

A second scale is the first test of whether the composition holds, so I ran F at 200M against block-entry mixing alone. F's two-seed mean is the lowest bpb of any 200M model, with the gain over block-entry mixing alone on every held-out set, while its retrieval moved more between seeds than between models (Figure 12; the values are in Table 3); Section 5 tells that story. Against the base, F's two-seed mean is 2.1% lower, a gap about 33 times the seed spread of the 200M runs. On harder recall sets, one headline cell of 37 passed my screen, about what chance alone would give: SQuAD completion, on which every run with block-entry mixing scored higher than the base and TST-only runs. On FDA, a field-extraction set, every run with block-entry mixing scored lower than the base and TST-only runs, but that cell's seed noise is large where its scores are low, so whether block-entry mixing costs this kind of lookup is open (Appendix D.4).

8.3 Longer training and other learning rates

My 60M runs are short, about 20 token positions per non-embedding parameter and well under one pass over the training set, so the question is whether F's lead lasts with more training. I continued the base, R1-entry and F from their ablation runs at a constant learning rate, with decayed branches at 1, 2, 4 and 8 times 20 token positions per parameter (all parameters counted), one seed per point. At equal compute, counted in matrix-multiply FLOPs where an update of F costs about 3% more, F stays below both the base and R1-entry at the base's 1x, 2x and 4x budgets, by a steady margin smaller than in the ablation runs (Figure 13); these values are placed between measured runs, without a noise verdict. Read the other way, the base needs 1.46, 1.52 and 1.72 times F's compute to reach the bpb of F's 1x, 2x and 4x branches (1.45 at the ablation runs), each value

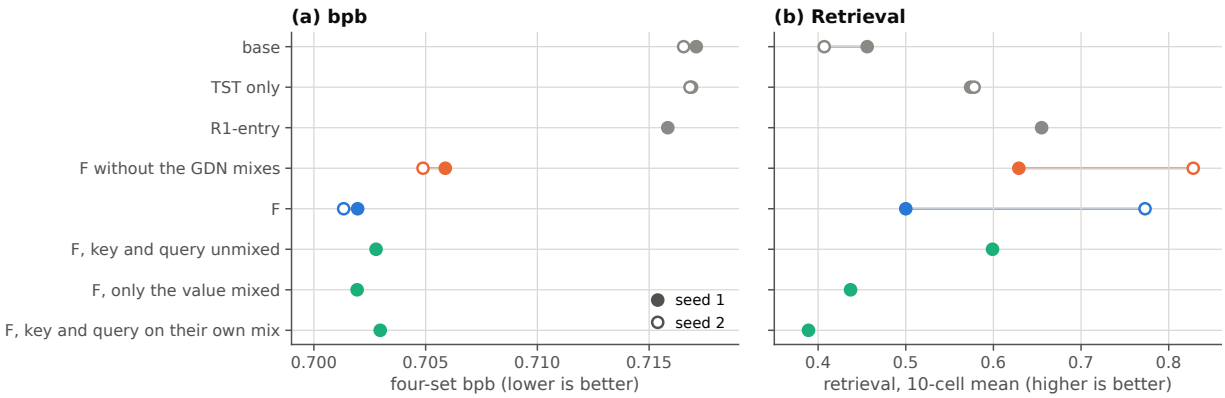


Figure 12: The 200M runs of Table 3, one dot per seed (filled: seed 1; open: seed 2). F's bpb lead over the base and over block-entry mixing alone is many times the spread between seeds; the retrieval differences are not: the two seeds of F lie 0.273 apart.

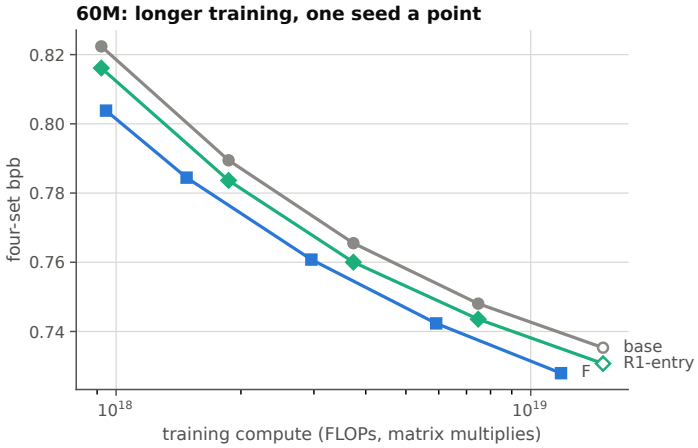


Figure 13: Four-set bpb against training compute at 60M, one seed per point: each model's ablation run and its branches at 1, 2, 4 and 8 times 20 token positions per parameter. Compute is positions times the matrix-multiply FLOPs per token (norms, gates, softmax and the depth mixes not counted). F is tied, so at each budget it trains fewer updates. Hollow markers: the two branches that read more than one pass of the training set. The ablation runs were trained on an earlier code snapshot than the branches, which resume from their checkpoints.

interpolated linearly in log compute between the two measured base points on either side; F's 8x branch lies below the base's last point and gets no value; like the gaps, these ratios rest on one seed per point and carry no noise verdict. bpb improved at every doubling for all three models, each step beyond the seed noise, while the ten-cell retrieval mean stayed within the seed noise from 1x to 8x for all three.

Table 8 gives the other ways of matching the runs, one line each here. At equal updates the lead is about the same as at equal compute. At the budget labels, which give the tied F fewer updates, it is smaller over the base and within the seed noise over R1-entry. At equal raw text, since TST reads four tokens per position early on, F is level with the base and behind R1-entry; I take equal compute as the comparison that matters, since equal raw text is only a small caveat about repeats

Table 8: F's bpb lead at 60M under each way of matching the runs (FINAL; one seed per point). Negative: F lower. "Placed" values are estimated between measured runs and carry no noise verdict.

Matched on	F− base	F− R1-entry
Equal compute at the base's 1x, 2x, 4x (placed)	−0.01299, −0.01095, −0.01055	−0.00716, −0.00546, −0.00607
The ablation runs (6,487 updates, seed 1)	−0.01856	−0.01230
Budget labels 1x, 2x, 4x, 8x	−0.00502, −0.00473, −0.00572, −0.00736	+0.00080, +0.00075, −0.00124, −0.00277
Equal updates at F's 1x (10,146; placed)	−0.0159 to −0.0171	−0.0099 to −0.0111
Equal updates at 26,301 to 81,162 (placed)	−0.01109 to −0.01225	−0.00615 to −0.00758
Equal raw text at F's 1x (placed)	−0.00044	+0.00532
Learning rate ×0.5, ×1, ×2	−0.02459, −0.01856, −0.01073	−0.02102, −0.01230, −0.00540

of code and mathematics, on which runs that read less than one pass say nothing. At half and at twice the standard learning rate F's bpb stayed below both others' beyond the seed noise (the standard-rate runs are the ablation runs, on an earlier code snapshot); the standard rate was best for the base and R1-entry, and for F the lowest rate was best, within the seed noise of the standard one. A run of F at a quarter of the standard rate, which tests whether its best rate lies below half the standard one, was queued at the time of writing (7 November 2026), with results to follow in a later version. Each point is one seed with the seed noise borrowed from the ablation set, and a flat retrieval mean does not show that retrieval stops improving.

9 Negative results

The copy-pointer head. A recent paper reports that with a copy head its language models reach the final loss of their plain counterparts up to twice as fast, and that at its smallest size the gains are 15 to 40 times the spread between seeds [8]. The head is a small pointer that scores the earlier positions of the context, and its copy distribution is mixed with the output head's through a learned gate, in the line of pointer-sentinel and pointer-generator models [26, 36]. I also wanted it because TST's gain had landed on rare tokens, which a pointer can copy instead of predict, and as a second, much less compressed gradient path. It gave by far the largest bpb gain of anything I tried, several times that of any piece I kept, and on the base trunk it also raised retrieval; but on the trunk with block-entry mixing retrieval collapsed to half of what the trunk had without the head, a check of the code found no fault, and the later forms I tried kept a bpb gain and also read lower on retrieval, by amounts partly within the run-to-run noise (Table 9). Training the head only as an auxiliary loss lost the gain, so the gain lives in the mixture at the output. I kept coming back to the head because its bpb gain was by far the largest of anything I tried, but in the jointly trained forms, where the pointer's gradient reaches the trunk, the trunk comes to lean on it, the add-on form, which stops that gradient (Appendix A), makes the output a mixture distribution that would get in the way of the reinforcement-learning and optimizer methods I meant to use in the main run, copying is something the trunk's own attention already learns, and I came to fear strongly that it would break down later in training, so I dropped it in every form.

Table 9: Negative results at 60M (FINAL): each arm's four-set bpb minus its trunk's (negative: lower, better) and its ten-cell retrieval mean with the trunk's in brackets, shown without a noise verdict (at two seeds a retrieval difference must exceed about 0.2 to stand out). The last row is the direct state read of Section 5. A step time in a row label is on my compiled but unfused code. –: not shown.

Arm	Trunk	Seeds	bpb	Retrieval
Copy-pointer head	base	1	−0.0351	0.370 (0.265)
Copy-pointer head	R1-entry	2	−0.0294	0.253 (0.517)
Copy-pointer head, seven later forms	R1-entry	2 each	lower	0.279 to 0.393 (0.517)
Copy-pointer head, auxiliary loss only	R1-entry	2	+0.0021	0.442 (0.517)
Negative eigenvalues	R1-entry	2	+0.00321	0.441 (0.517)
Per-channel decay	R1-entry	2	−0.00116	0.384 (0.517)
Per-channel decay with a floor	R1-entry	2	−0.00018	0.442 (0.517)
Two delta steps per token (step ×1.24)	R1-entry	2	+0.00032	0.579 (0.517)
Separate erase direction	R1-entry	2	−0.00074	0.388 (0.517)
Reading the state of the layer below; with a translator	R1-entry	1	−0.0002; +0.0002	–

Delta-rule variants. Many GDN heads keep almost no memory (at 60M with block-entry mixing, 9 to 10 of 60 heads had a median memory under two tokens), and TST brought many of them back, so I hoped an update rule that keeps memory alive by design would give TST's retrieval gain without its extra data. In a diagnostic on the 200M TST models, forcing the revived heads back to short memory removed only about a tenth of TST's retrieval gain, so that premise fell first, but I screened the variants anyway, two seeds each at 60M: negative eigenvalues [9] cost bpb; per-channel decay [19], with or without a floor on part of the channels' memory, was level on bpb and read lower on retrieval; two delta steps per token [40] raised retrieval, at twice the state updates per token and about a quarter more step time on my compiled but unfused code; and a separate erase direction [22] and cheaper ways to get the second step all read worse on bpb or level; at two seeds none of these retrieval differences reaches the 0.2 that would stand out from the run-to-run noise. None of 15 trained arms paid for its cost in bpb, and plain Gated DeltaNet stays; two of them, negative eigenvalues and two delta steps per token, are proposed for state tracking [9, 40], which a 60M bpb screen does not measure, so the screen says only that these rules do not pay for themselves in my recipe at this size.

Grid sharing with less cache. At 200M, a 2×4 grid with the cache of three KV heads trailed four KV heads, two seeds each: its bpb was higher in both seeds, inside the spread; its 16-key retrieval at 8k, 16k and 32k fell below the control's beyond each cell's own bar; and its FDA scores were lower in both seeds (Section 6.3). Grid sharing redistributes sharing at equal cache; it does not replace cache.

10 Other small experiments

A drafter for speculative decoding. A multi-token-prediction head [7] trained on a frozen 60M base model, as a drafter for speculative decoding [21], decoded exactly as greedy decoding does and committed several tokens per verification step. Its 200M tests waited for the main run's compute and were dropped with that plan.

Table 10: Small screens at 60M (FINAL): each arm's four-set bpb minus its trunk's (negative: lower, better). The first two ran on an earlier looped trunk (Appendix C), before Scalable-Softmax.

Arm	Trunk	Seeds	bpb
128k vocabulary	earlier	2	+0.0306
Squared ReLU	earlier	2	+0.0000
Feed-forward 3.6× wider, fewer layers: equal tokens (equal time)	base	1	+0.0037 (−0.0016)
GDN key halved	base	1	+0.0013
Canon layers, two starts	base	1	−0.0008, −0.0013

Small screens. Screens at 60M left several defaults as they were (Table 10). On an earlier trunk, a 128k vocabulary was worse than my 64k and squared ReLU was level with SwiGLU, verdicts that stayed provisional because the copy and edit probes were never added to them. On the base, a wider, shallower feed-forward was worse at equal tokens and better at equal time, which I judged would not carry over to the 4B; a halved GDN key and Canon layers [2] gave no gain beyond the seed noise.

11 Limitations

Everything here comes from models of 60M and 200M parameters with few seeds: three per model in the final 60M ablation set, one or two elsewhere, and one or two at 200M. My retrieval probes are synthetic lookups scored by exact match, retrieval's run-to-run noise hides most effects at two seeds, and at 200M the runs with TST repeated more of their data than the others did. I give costs as operation counts, as step times on my compiled but unfused code at 60M, and, for the write mix and the block-entry reads, as fused kernels that were not timed against their controls in one job; none was measured with the kernels or on the hardware a larger model would train on.

12 Background of this work

I set out to train a small model from scratch and tried every part of it. That is why this report covers so many pieces, including methods that are not mine and lines I closed: each was a question I had to answer for the model I wanted to build. I had no GPUs for the main training run, so the work stopped at these small models. What is left is four ideas tested as far as models of 60M and 200M parameters allow, and, for each, the questions that would take it further.

Acknowledgements

AI coding agents were used to run the experiments and to draft the manuscript. All design decisions are the author's, and the author takes full responsibility for the content.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training Generalized Multi-Query Transformer Models from Multi-Head

Checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023. URL https://arxiv.org/abs/2305.13245.

[2] Zeyuan Allen-Zhu. Physics of Language Models: Part 4.1, Architecture Design and the Magic of Canon Layers, 2025. URL https://arxiv.org/abs/2512.17351.

[3] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, Dylan Zinsley, James Zou, Atri Rudra, and Christopher Ré. Simple linear attention language models balance the recall-throughput tradeoff, 2024. URL https://arxiv.org/abs/2402.18668. Published at ICML 2024, PMLR 235:1763–1840.

[4] Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. Universal Transformers. In *International Conference on Learning Representations (ICLR)*, 2019. URL https://arxiv.org/abs/1807.03819.

[5] Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach, 2025. URL https://arxiv.org/abs/2502.05171.

[6] Gleb Gerasimov, Yaroslav Aksenov, Nikita Balagansky, Viacheslav Sinii, and Daniil Gavrilov. You Do Not Fully Utilize Transformer's Representation Capacity, 2025. URL https://arxiv.org/abs/2502.09245. Accepted to EMNLP 2026 (Main Conference).

[7] Fabian Gloeckle, Badr Youbi Idrissi, Baptiste Rozière, David Lopez-Paz, and Gabriel Synnaeve. Better & Faster Large Language Models via Multi-token Prediction, 2024. URL https://arxiv.org/abs/2404.19737.

[8] Nathan Godey and Yoav Artzi. Copy-Paste Language Models, 2026. URL https://arxiv.org/abs/2610.19247.

[9] Riccardo Grazzi, Julien Siems, Arber Zela, Jörg K.H. Franke, Frank Hutter, and Massimiliano Pontil. Unlocking State-Tracking in Linear RNNs Through Negative Eigenvalues, 2024. URL https://arxiv.org/abs/2411.12537. Published at ICLR 2025.

[10] Wei He, Kai Han, Yehui Tang, Chengcheng Wang, Yujie Yang, Tianyu Guo, and Yunhe Wang. DenseMamba: State Space Models with Dense Hidden Connection for Efficient Large Language Models, 2024. URL https://arxiv.org/abs/2403.00818.

[11] Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. RULER: What's the Real Context Size of Your Long-Context Language Models?, 2024. URL https://arxiv.org/abs/2404.06654. Published at COLM 2024.

[12] Jingcheng Hu, Houyi Li, Yinmin Zhang, Zili Wang, Shuigeng Zhou, Xiangyu Zhang, Heung-Yeung Shum, and Daxin Jiang. Multi-matrix Factorization Attention, 2024. URL https://arxiv.org/abs/2412.19255. Published in Findings of the Association for Computational Linguistics: ACL 2025, pages 25114–25126.

[13] Hugging Face. Mistral Large 4. Hugging Face Transformers documentation, 2026. URL https://huggingface.co/docs/transformers/main/en/model_doc/mistral_large4. Model released October 2026; accessed 2026-11-03.

[14] Hakan Inan, Khashayar Khosravi, and Richard Socher. Tying Word Vectors and Word Classifiers: A Loss Framework for Language Modeling. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://openreview.net/forum?id=r1aPbsFle.

[15] Keller Jordan, Yuchen Jin, Vlado Boza, Jiacheng You, Franz Cesista, Laker Newhouse, and Jeremy Bernstein. Muon: An optimizer for hidden layers in neural networks, 2024. URL https://kellerjordan.github.io/posts/muon/.

[16] Keller Jordan et al. modded-nanogpt: the NanoGPT speedrun. GitHub repository, 2024. URL https://github.com/KellerJordan/modded-nanogpt. Accessed 2026-11-05.

[17] Jeonghoon Kim, Byeongchan Lee, Cheonbok Park, Yeontaek Oh, Beomjun Kim, Taehwan Yoo, Seongjin Shin, Dongyoon Han, Jinwoo Shin, and Kang Min Yoo. Peri-LN: Revisiting Normalization Layer in the Transformer Architecture. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.02732.

[18] Kimi Team. Attention Residuals, 2026. URL https://arxiv.org/abs/2603.15031.

[19] Kimi Team, Yu Zhang, Zongyu Lin, Xingcheng Yao, et al. Kimi Linear: An Expressive, Efficient Attention Architecture, 2025. URL https://arxiv.org/abs/2510.26692.

[20] Guillaume Lample, Alexandre Sablayrolles, Marc'Aurelio Ranzato, Ludovic Denoyer, and Hervé Jégou. Large Memory Layers with Product Keys. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019. URL https://arxiv.org/abs/1907.05242.

[21] Yaniv Leviathan, Matan Kalman, and Yossi Matias. Fast Inference from Transformers via Speculative Decoding. In *International Conference on Machine Learning (ICML)*, 2023. URL https://arxiv.org/abs/2211.17192.

[22] Xiao Li, Chengruidong Zhang, Hao Luo, Xi Lin, Zekun Wang, Zihan Qiu, Yunfei Mao, Langshi Chen, Man Yuan, Minmin Sun, Huiqiang Jiang, Siqi Zhang, Rui Men, Wei Hu, Gong Cheng, Bo Zheng, Dayiheng Liu, and Jingren Zhou. Erase-then-Delta Attention: Decoupling Erase and Write Addresses in Delta-Rule Linear Attention, 2026. URL https://arxiv.org/abs/2606.26560.

[23] Opher Lieber, Barak Lenz, Hofit Bata, Gal Cohen, Jhonathan Osin, et al. Jamba: A Hybrid Transformer-Mamba Language Model, 2024. URL https://arxiv.org/abs/2403.19887.

[24] Xiaoran Liu, Hang Yan, Shuo Zhang, Chenxin An, Xipeng Qiu, and Dahua Lin. Scaling Laws of RoPE-based Extrapolation. In *International Conference on Learning Representations*, 2024. URL https://arxiv.org/abs/2310.05209.

[25] Cheng Luo, Zefan Cai, and Junjie Hu. Multi-Head Attention Residuals, 2026. URL https://arxiv.org/abs/2607.27230.

[26] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer Sentinel Mixture Models. In *International Conference on Learning Representations (ICLR)*, 2017. URL https://arxiv.org/abs/1609.07843.

[27] Ken M. Nakanishi. Scalable-Softmax Is Superior for Attention, 2025. URL https://arxiv.org/abs/2501.19399.

[28] Bahareh Nikpour, Hossein Rajabzadeh, Maryam Dialameh, Vahid Partovi Nia, and Parsa Omidi. Kronecker Product Attention: Memory-Efficient KV Compression via Structured Factorization, 2026. URL https://arxiv.org/abs/2610.12657. NeurIPS 2026 Workshop on SLM-Agents.

[29] nktkt. attention-residuals: PyTorch implementation of Attention Residuals (arXiv:2603.15031). GitHub repository, 2026. URL https://github.com/nktkt/attention-residuals. Created 2026-03-24.

[30] James O'Neill, Robert Clancy, Mariia Matskevichus, and Fergal Reid. Low-Rank Key Value Attention, 2026. URL https://arxiv.org/abs/2601.11471.

[31] Matteo Pagliardini, Amirkeivan Mohtashami, François Fleuret, and Martin Jaggi. DenseFormer: Enhancing Information Flow in Transformers via Depth Weighted Averaging. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. URL https://arxiv.org/abs/2402.02622.

[32] Bowen Peng, Théo Gigant, and Jeffrey Quesnelle. Efficient Pre-Training with Token Superposition, 2026. URL https://arxiv.org/abs/2605.06546.

[33] Ofir Press and Lior Wolf. Using the Output Embedding to Improve Language Models. In *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics (EACL), Volume 2, Short Papers*, pages 157–163, 2017. URL https://arxiv.org/abs/1608.05859.

[34] Qwen Team. Qwen3-Next-80B-A3B-Instruct. Hugging Face model card, 2025. URL https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct.

[35] Qwen Team. On the Design of Qwen3.8-Next Architecture: Evaluation, Efficiency, and Training Stability, 2026. URL https://arxiv.org/abs/2608.30320.

[36] Abigail See, Peter J. Liu, and Christopher D. Manning. Get To The Point: Summarization with Pointer-Generator Networks. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1073–1083, Vancouver, Canada, 2017. URL https://aclanthology.org/P17-1099/.

[37] Chenze Shao, Fandong Meng, and Jie Zhou. Beyond Next Token Prediction: Patch-Level Training for Large Language Models. In *International Conference on Learning Representations (ICLR)*, 2025. URL https://arxiv.org/abs/2407.12665.

[38] Noam Shazeer. GLU Variants Improve Transformer, 2020. URL https://arxiv.org/abs/2002.05202.

[39] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism, 2019. URL https://arxiv.org/abs/1909.08053.

[40] Julien Siems, Timur Carstensen, Arber Zela, Frank Hutter, Massimiliano Pontil, and Riccardo Grazzi. DeltaProduct: Improving State-Tracking in Linear RNNs via Householder Products, 2025. URL https://arxiv.org/abs/2502.10297. Published at NeurIPS 2025.

[41] Jianlin Su. Transformer Upgrade Road: 12. ReRoPE for Infinite Extrapolation? Scientific Spaces (kexue.fm), August 2023. URL https://kexue.fm/archives/9708. Blog post (in Chinese; title translated), 2023-08-07.

[42] Jianlin Su. Transformer Upgrade Road: 9. A New Idea for Global Length Extrapolation. Scientific Spaces (kexue.fm), May 2023. URL https://kexue.fm/archives/9603. Blog post (in Chinese; title translated), 2023-05-12.

[43] Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. RoFormer: Enhanced Transformer with Rotary Position Embedding, 2021. URL https://arxiv.org/abs/2104.09864.

[44] Da Xiao, Qingye Meng, Shengping Li, and Xingyuan Yuan. MUDDFormer: Breaking Residual Bottlenecks in Transformers via Multiway Dynamic Dense Connections. In *International Conference on Machine Learning (ICML)*, 2025. URL https://arxiv.org/abs/2502.12170.

[45] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated Delta Networks: Improving Mamba2 with Delta Rule, 2024. URL https://arxiv.org/abs/2412.06464.

[46] Ted Zadouri, Hubert Strauss, and Tri Dao. Hardware-Efficient Attention for Fast Decoding, 2025. URL https://arxiv.org/abs/2505.21487.

[47] Yifan Zhang, Yifeng Liu, Huizhuo Yuan, Zhen Qin, Yang Yuan, Quanquan Gu, and Andrew Chi-Chih Yao. Tensor Product Attention Is All You Need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2025. URL https://arxiv.org/abs/2501.06425.

A Noise statistics

The bar of 2.87. I call a difference beyond the seed noise when it exceeds 2.87 of its standard errors, estimated from the spread between seeds. I chose 2.87 early, when noise was estimated from three seeds. A plain two-sigma rule calls a difference that is not there "clear" 4.6% of the time when the standard deviation is known, but 18% of the time when it comes from three seeds, with 2 degrees of freedom. My bar uses 4 degrees of freedom, the spread pooled over the two models compared, where a plain two-sigma rule would still give 11.6%; 2.87 is the Student-t value at 4 degrees of freedom whose two-sided tail is the normal's 4.6%. I kept 2.87 as a fixed number afterwards, also where the spread rests on more degrees of freedom. In the final 60M ablation set the pooled standard deviation of a model's bpb across seeds is 0.00146 (20 degrees of freedom), so with three seeds per model a bpb difference of about 0.0034 clears the bar.

Retrieval's run-to-run noise. Early on I estimated retrieval's seed noise from a few seeds, at 0.0288 for the ten-cell mean, and several early retrieval verdicts in my records used it. An add-on form of the copy-pointer head (Section 9), meant to stop every gradient from the pointer into the trunk, then lost retrieval inside a stack. A check that read the code paths and ran a small model on a CPU found no gradient path from the add-on into the trunk. The trained trunks of the stack with and without the add-on still drifted apart from the second update, for a reason the records do not settle. That sent me to measure the run-to-run variance over many pairs of runs: the standard deviation of the ten-cell mean is 0.0760 (38 degrees of freedom), more than twice the earlier figure, and against it the add-on's retrieval loss lies inside the measured spread. With it, a retrieval difference must exceed 0.308 to clear the bar with one run of each model, and 0.218

with two of each. At 200M the spread measured between the second seeds is 0.1207 (4 degrees of freedom; 95% interval 0.0723 to 0.3468), 1.59 times the 60M figure. For a retrieval difference d to clear 2.87 standard errors with n seeds per model and run-to-run standard deviation σ, n must be at least $2\,(2.87\,\sigma/d)^2$; the seed counts in the open questions of Sections 3 and 5 come from this, with $\sigma = 0.1207$ at 200M. Because that σ rests on 4 degrees of freedom, each count could be several times smaller or larger over the 95% interval of σ.

The recall screen at 200M. The harder recall sets of Section 2 (Appendix D.4) were screened before any result entered the main table. For each cell, the signal is the standard deviation of the four configurations' means (base, TST only, F, F without the GDN mixes, two seeds each) and the noise is the pooled seed standard deviation of a configuration's mean; a cell enters the main table when their ratio is at least 2.87. With four configurations of two seeds the squared ratio is a one-way F statistic on 3 and 4 degrees of freedom, and, if every configuration has the same seed variance, a cell whose configurations do not differ passes with probability 0.0347; of 37 headline cells, about 1.3 would pass by chance, and one passed. That one cell does not meet the assumption: its largest seed spread is 11 times its smallest. A pass is a screen for the main table, not a test that the configurations differ.

B Data sources and licences

All training sources are public and ungated, downloaded without a token, and pinned to a revision. Table 11 lists them with the licence on each card at that revision. Some cards add terms: the DCLM card says the set is "intended for research use only", and the card of one science set says a distributed model may be subject to the Qwen, DeepSeek and Phi-4 terms.

Table 11: Training sources (and the general-text evaluation source).

Category	Source (Hugging Face)	Licence on the card
Web	HuggingFaceFW/fineweb-edu	ODC-By 1.0 and Common Crawl terms
Web	mlfoundations/dclm-baseline-1.0-parquet	CC-BY-4.0
PDF text	HuggingFaceFW/finepdfs-edu (English)	ODC-By 1.0 and Common Crawl terms
Code	common-pile/stackv2_edu_filtered	per-file permissive licences; Stack v2 terms
Mathematics	HuggingFaceTB/finemath (finemath-4plus)	ODC-By 1.0 and Common Crawl terms
Mathematics	tokyotech-llm/swallow-math-v2	Apache-2.0
Science	nvidia/Nemotron-Pretraining-Specialized-v1	CC-BY-4.0 per row
Agent	nvidia/Open-SWE-Traces	CC-BY-4.0; task repositories MIT, Apache-2.0, BSD
Agent	Agent-Ark/Toucan-1.5M (SFT)	Apache-2.0
Agent	nvidia/Nemotron-SFT-Agentic-v2	CC-BY-4.0 (the card also lists Apache-2.0, MIT)
General (evaluation only)	wikimedia/wikipedia (English)	CC-BY-SA-3.0 and GFDL

In three of its six tasks, the RULER subset of Section 2 hides its needles in my own held-out web and Wikipedia documents, because RULER's essay haystack has no open licence I could find; the

other three keep RULER's own haystacks; its numbers are therefore not comparable with published RULER numbers.

C Looped models

Running a shared block of layers more than once buys depth without parameters [4, 5]. Under a cap of 4.0B parameters that is attractive, so I started from a looped design: a causal encoder–decoder in which every token passed once through an encoder that wrote a memory of keys and values, and a shared decoder block ran twice at the positions where the model produces output. I expected extra depth where the model has to decide, and a cheaper read of the many tokens an agent only reads.

What happened. At 60M the looped design lost clearly to a plain 3:1 hybrid of the same size at equal wall-clock: 0.889 against 0.822 bpb. A form with the decoder on every position and a single pass recovered 87% of that gap, so in my reading most of the loss came from the training recipe rather than the looped structure. But the second pass never paid for its compute: each of the six looped arms of the encoder–decoder design was 0.004 to 0.007 bpb worse than the same model with one pass at equal wall-clock, and on the plain hybrid a second pass cost 0.0043 bpb over the top eight layers and 0.0026 over the top four (one seed each). A re-test built the way recent looped models are, on synthetic reasoning tasks, showed no gain either, and those tasks turned out to measure how many documents a model has seen: a plain 20-layer model learned them in one pass once it had seen enough.

Why I set it aside. With the parameters capped and the compute fixed, a loop can only buy depth with compute that would otherwise train on more tokens, and at 60M that compute always did better spent on the plain hybrid. I set loops aside for this model for those two reasons, size and compute. This is a screen on bpb at 60M, not a verdict on looping, and no loop was trained at 200M.

D Further details

D.1 The causality check

Depth mixes and token bags move information along paths a plain transformer lacks, and a leak of future tokens along any of them would look like a large gain. So F's training path was checked: changing every token after a position must leave the outputs up to that position unchanged, and the check caught all five leaks planted on purpose. On dense attention the earlier outputs were exactly unchanged; on the fast bf16 kernel a few moved by less than the model's own bf16 rounding error, from the order in which the kernel adds its tiles.

D.2 Key–value heads before the grid

My small models all use one KV head. A first 60M test of two and four KV heads kept the parameter count equal by narrowing every feed-forward layer, and bpb did not improve in one seed each; those runs do not isolate the KV heads, and in my reading their bpb loss came mostly from the narrower feed-forward layers, not from the KV heads. A 200M run with heads of size 256 had twice the KV width and, in one seed, a retrieval gain too small to stand out from the noise (t +1.93), so it could not separate head size from width. Two KV heads at full width on the trunk with block-entry

mixing were 0.0013 bpb worse over two seeds, but the whole gap is in seed 1 and seed 2 shows about none, so the two seeds disagree. At 200M, two KV heads against one, two seeds each, split between the seeds on bpb and read the same on retrieval; four KV heads had lower bpb than two outside the spread, in both seeds, and lower than one by an amount inside the spread (Section 6). The 4B draft configuration's 8 KV heads were carried over from an earlier sizing and never chosen by an experiment.

D.3 The 60M ablation set and the stacks

Table 12 gives the values behind Section 8.1: part (a) the final ablation set of Figure 11, part (b) the earlier stacks and the query mix with TST of Section 5. In part (b), a piece's contribution inside a stack is the stack's bpb without the piece minus the stack's bpb; alone, it is the piece's gain over block-entry mixing.

Table 12: The 60M ablation set and the stacks (FINAL). (a) The final ablation set: three seeds per model, one code snapshot; four-set bpb (lower is better) and the ten-cell retrieval mean (higher is better), with each model's difference from F; †: beyond the seed noise. Leaving out block-entry mixing also removes the GDN mixes, which read its sources. (b) The stacks on the trunk with block-entry mixing, two seeds each: a positive bpb gain means a lower bpb; retrieval is the change in the ten-cell mean that the stack or the piece brings.

(a) The final ablation set

Model	bpb	−F	Retrieval	−F
F	0.80401		0.5212	
Base	0.82225	+0.01823†	0.2653	−0.2559†
Block-entry mixing only	0.81727	+0.01326†	0.5007	−0.0206
F without block-entry mixing	0.81591	+0.01189†	0.3177	−0.2036†
F with 3:2 instead of 4:1	0.80381	−0.00020	0.4580	−0.0632
F without the query mix	0.80403	+0.00001	0.4832	−0.0380
F without both GDN mixes	0.80636	+0.00235	0.5903	+0.0691
F without TST	0.80865	+0.00463†	0.4791	−0.0421
F untied	0.80650	+0.00249	0.4650	−0.0562

(b) The stacks on block-entry mixing

	bpb gain	Retrieval
Write mix, TST and 4:1 together	0.00986	−0.042
Write mix and TST, without 4:1	–	−0.084
write mix: inside the stack, alone	0.00335, 0.00268	
TST: inside the stack, alone	0.00688, 0.00635	
4:1: inside the stack, alone	0.00005, −0.00015	
Copy-pointer head inside the five-piece stack (alone)	0.02436 (0.03347)	−0.211
F (sum of its parts alone)	0.01294 (0.01437)	
Query mix and TST together	–	0.4842[a]

[a]The ten-cell mean itself, not a change: the trunk reads 0.5168, the query mix alone 0.5235 and TST alone 0.5735.

D.4 Recall evaluations at 200M

All twelve 200M finals were also scored on the harder recall sets of Section 2, and only cells whose spread between configurations is large against their seed noise enter the main table (Appendix A). Of 37 headline cells one passed, about what chance alone would give: SQuAD completion, on which

every run with block-entry mixing scored higher than the base and TST-only runs (Table 13). Every multi-key and RULER-subset cell fell below the bar, and so did SWDE and FDA. FDA asks for the value of a named field in a document of about 2,000 tokens. On it the base and TST-only runs scored about the same in both seeds, while every run with block-entry mixing scored lower, and nearly half of R1-entry's extra misses are the value of another field of the same document. But the cell's seed noise is large exactly where its scores are low: the two seeds of F, and those of F without the mixes, differ far more than those of the base and TST only, so whether block-entry mixing costs this kind of lookup is open.

Table 13: Two recall cells at 200M, accuracy by seed (FINAL): SQuAD completion, the one headline cell that passed the screen, and FDA, which did not. Every run but the base and TST only has block-entry mixing.

Run	SQuAD completion	FDA
Base	0.36212, 0.3577	0.75, 0.74909
TST only	0.36892, 0.35396	0.76818, 0.76818
R1-entry	0.38694	0.63364
F	0.3934, 0.39476	0.65545, 0.73273
F without the GDN mixes	0.38422, 0.39204	0.58455, 0.67818
F, key and query unmixed	0.40292	0.71
F, only the value mixed	0.40326	0.64091
F, key and query on their own mix	0.39544	0.66818